AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ INŻYNIERII MECHANICZNEJ I ROBOTYKI

KATEDRA ROBOTYKI

Praca dyplomowa inżynierska

Badanie protokołów komunikacyjnych w systemach multiplatform.

The study of communication protocols in multiplatform systems.

Autor: *Antoni Dziurdzia*
Kierunek studiów: Automatyka Przemysłowa i Robotyka
Opiekun pracy: *dr hab. inż. Adam Jabłoński, prof. AGH*

Kraków, 2024

1. Wstęp ... 3
1.1. Motywacja ... 3
1.2. Cel ... 3
1.3. Zakres ... 3
2. Przegląd i porównanie najpopularniejszych systemów wbudowanych ... 6
2.1. Wstęp ... 6
2.2. ESP32 ... 6
2.3. STM32 ... 8
2.4. Raspberry Pi ... 11
2.5. NVIDIA Jetson ... 13
2.6. Wybór platformy wbudowanej ... 14
3. Wstęp do badania protokołów ... 16
3.1. Protokół ... 16
3.2. Interfejs ... 16
3.3. Model OSI ... 16
4. Protokoły komunikacyjne – projekty z implementacją ... 17
4.1. Protokoły sieciowe ... 17
4.2. Protokoły szeregowe ... 42
4.3. Protokoły wysokiego poziomu ... 51
5. Protokoły komunikacyjne – opracowania teoretyczne ... 60
5.1. Protokoły przemysłowe ... 60
5.2. Protokoły szeregowe niskiego poziomu ... 63
5.3. Protokoły sieciowe ... 65
6. Technologie sieciowe ... 66
6.1. Interfejsy ... 66
6.2. Firewall ... 66
7. Podsumowanie ... 67

1. Wstęp

1.1. Motywacja

Przedmiotem tej pracy inżynierskiej jest badanie protokołów w systemach multiplatform. Poznanie zasad działania protokołów, oraz umiejętność wykorzystania ich w praktyce otwiera szeroki zakres perspektyw zawodowych. Umiejętność wdrażania a także rozumienie protokołów przydaje się w automatyce przemysłowej, telekomunikacji oraz w informatyce, gdzie komunikacja oraz interoperacyjność urządzeń odgrywa coraz większą rolę.

1.2. Cel

Celem pracy jest: poznanie podstawowych protokołów wykorzystywanych w automatyce przemysłowej, internecie rzeczy i sieci internet oraz poznanie popularnych systemów wbudowanych. Praktycznym celem jest opracowanie dziewięciu projektów wykorzystujących wybrane protokoły, których zadaniem jest komunikacja procesów na różnych urządzeniach.

1.3. Zakres

Zakres pracy obejmuje przegląd i porównanie najpopularniejszych systemów wbudowanych, przedstawienie podstawy teoretycznej protokołu oraz szczegółowe opracowanie 15 protokołów, z których dziewięć jest popartych projektem. Pozostałe 6 protokołów zawiera jedynie opracowanie teoretyczne ze względu na ograniczenia sprzętowe oraz czasowe. W ostatniej sekcji omówiono 2 technologie sieciowe. Do realizacji części praktycznej wykorzystano RaspberryPi model 4B.

Tabela 1.1*: Protokoły sieciowe*

Protokół	Opis	Projekt	Logo
HTTP	Protkół tekstowy wykorzystywany w sieciach www	Tak	
TCP	Niezawodny protokół warstwy transportowej, podstawa internetu	Tak	
FTP	Protokół transferu plików	Tak	

MQTT	Lekki protokół oparty o wzorzec publikacja/subskrypcja	Tak	
Modbus TCP	Protokół modbus wykorzystujący ethernet	Tak	
WebSockets	Nowczesny protokół łączący klienta z serwerem w sposób ciągły	Nie	

W tabeli 1.1 przedstawiono protokoły sieciowe zbadane w przebiegu pracy inżynierskiej. W tabeli obok nazwy protokołu znajduje się krótki opis, informacja o tym czy badany protokół został oparty projektem oraz logo. W tabeli 1.1 mamy protokoły warstwy aplikacji oraz transportowej, które wyróżniają się szerokim zastosowaniem w internecie, internecie rzeczy (IoT) oraz sieciach lokalnych.

Tabela 1.2*: Protokoły szeregowe*

Protokół	Opis	Projekt	Logo
Bluetooth	Umożliwia bezprzewodową ktrótkodystansową wymianę danych	Tak	
CAN	Magistrala wykorzystywana głównie w przemyśle motoryzacyjnym	Tak	
I2C	Protokół oparty na magistrali szeregowej, wykorzystywany w elektronice I w systemach wbudowanych	Nie	

SPI	Szybki protokół wykorzystywany w komunikacji mikrokontrolera z urządzeniami peryferyjnymi	Nie	

W tabeli 1.2 przedstawiono są protokoły bazujące na szeregowej wymianie danych. Bluetooth to technologia bezprzewodowa, pozwalająca na wymianę danych na niewielkich odległościach. Protokół CAN to magistrala wykorzystywana w przemyśle motoryzacyjnym. Protokoły i2c oraz SPI to protokoły wykorzystywane na mikrokontrolerach, służą do wymiany danych na niewielkich odległościach i nie zostały poparte projektem ze względu na ograniczenie czasowe. Osobiście największe wrażenie wywarł na mnie protokół CAN, który w swojej prostocie idealnie nadaje się do wykorzystania w motoryzacji, która jest moim zainteresowaniem.

Tabela 1.3*: Protokoły przemysłowe*

Protokół	Opis	Projekt	Logo
OPC UA	Protokół wysokiego poziomu mający na celu ujednolicenie komunikacji w fabrykach	Tak	
Profibus	Protokół wykorzystywany przez sterowniki PLC	Nie	
Profinet	Nowsza wersja Profibusa wykorzstująca Ethernet jako medium transmisyjne	Nie	
Modbus RTU	Protokół przemysłowy wykorzystujący interfejs rs-485	Nie	

W Tabeli 1.3 wymieniono protokoły wykorzystywane w automatyce przemysłowej. Służą do sterowania procesami, i najczęściej wykorzystuje się je w połączeniach maszyn lub linii produkcyjnych ze sterownikami. Warto zwrócić uwagę na protokół OPC UA, który jest próbą ujednolicenia komunikacji w fabrykach. Protokoły Profibus, Profinet i ModbusRTU to protokoły służące do komunikacji ze sterownikami, więc nie zostały one oparte o projekt na RaspberryPi.

2. Przegląd i porównanie najpopularniejszych systemów wbudowanych

2.1. Wstęp

System wbudowany (ang. embabed system) jest to system komputerowy będący częścią większego systemu i wykonujący istotną część jego funkcji [3]. Każdy system wbudowany oparty jest na mikroprocesorze(lub mikrokontrolerze), który zaprogramowany jest do wykonywania tylko kilku a czasem nawet jednego zadania. Zależnie od przeznaczenia może zawierać oprogramowanie przeznaczone jedynie na to urządzenie lub system operacyjny. Zwykle decyduje o tym stopień niezawodności jaki jest wymagany od systemu. Im mniej złożone i specjalizowane jest oprogramowanie tym system jest bardziej niezawodny, i szybciej reaguje na zdarzenia krytyczne.

Obecnie systemy wbudowane znajdują zastosowanie prawie we wszystkich dziedzinach, gdyż dąży się do tego żeby wszystkie urządzenia były „inteligentne" i zdolne do pracy autonomicznej, oraz do wykonywania coraz to bardziej złożonych zadań [1].

Przykłady obszarów i urządzeń technicznych, w których stosuje się systemy wbudowane:

- Układy sterujące pracą silnika samochodowego i ABS, komputery pokładowe.
- Sprzęt sterujący samolotami, rakietami
- Sprzęt medyczny
- Sprzęt pomiarowy, w tym m.in. oscyloskopy, analizatory widma
- Sterowniki PLC
- Systemy sterujące robotów mechanicznych
- Kuchenki mikrofalowe, zmywarki, itp.

2.2. ESP32

ESP32 jest wyjątkową płytą rozwojową, dzięki swojej wszechstronności i zaawansowanej architekturze, oferuje wiele funkcji i możliwości, które umożliwiają skuteczną i precyzyjną pracę.

Płyta ESP32 jest wyposażona w dwurdzeniowy procesor taktowany zegarem o częstotliwości do 240 MHz, co zapewnia dobrą moc obliczeniową do przetwarzania danych z czujników. Dodatkowo, ESP32 posiada wbudowaną pamięć Flash i SRAM, co umożliwia przechowywanie programów oraz danych bezpośrednio na płycie. ESP32 oferuje bogaty zestaw interfejsów komunikacyjnych, w tym UART, SPI i I2C. Interfejsy te umożliwiają integrację z peryferiami i modułami, takimi jak wyświetlacze, karty SD czy moduły komunikacji bezprzewodowej. ESP32 obsługuje również technologie bezprzewodowe, w tym WiFi i Bluetooth. To otwiera możliwość przesyłania danych bezprzewodowo do innych urządzeń lub do chmury [1]. Możliwość komunikacji przez Bluetooth pozwala również na sterowanie platformą zdalnie z urządzeń mobilnych.

Jedną z największych zalet ESP32 jest również jego obszerna społeczność programistów i dostępność różnych narzędzi programistycznych oraz bibliotek. Dzięki temu programowanie

platformy wbudowanej opartej na ESP32 jest znacznie ułatwione, a tworzenie aplikacji związanych z komunikacją opartą na protokołach staje się prostsze i bardziej efektywne.

Tabela 2.1*: Parametry techniczne układu ESP32*

	ESP32
CPU	CoreMark 2-core 32-bit
CPU FREQ	204 Mhz
RAM	520 KB
FLASH	4 MB
I/O	34
ADC	2x 8bit
Timer	2x 64bit
DMA	1x12
USART	2
SPI	4
I2C	2
Wi-Fi	802.11n(2.4GHz) do 150Mbps
Bluetooth	V4.2 oraz BLE
Ethernet	Brak
SD Slot	0
Zużycie energii	95-240 uA

W tabeli 2.1 przedstawiono parametry techniczne układu wbudowanego ESP32. Układ ten jest popularnym wyborem do budowy platform wbudowanych, charakteryzuje się szeregiem kluczowych parametrów. Jest to dwurdzeniowy procesor 32-bitowy, który oferuje wsparcie dla wielu funkcji i zapewnia dobrą moc obliczeniową. Częstotliwość procesora wynosi 204 MHz, co przekłada się na wydajność działania. Układ ten posiada 520 KB pamięci RAM, umożliwiającej przechowywanie danych w trakcie działania. Dodatkowo, wyposażony jest w 4 MB pamięci Flash, służącej do przechowywania programów i danych. ESP32 oferuje 34 piny I/O, co umożliwia podłączenie różnych urządzeń i czujników. W zakresie akwizycji danych, posiada dwa wejścia analogowo-cyfrowe (ADC) o rozdzielczości 8 bitów. Dzięki temu jest w stanie mierzyć

przyspieszenie, nachylenie oraz wibracje lub wstrząsy. Układ ESP32 jest wyposażony w dwa 64-bitowe liczniki czasu (timery), które mogą być wykorzystane do precyzyjnego pomiaru czasu. Posiada również jeden kanał DMA, umożliwiający efektywny transfer danych między różnymi urządzeniami. W zakresie komunikacji, oferuje dwa interfejsy I2C. dwa interfejsy USART oraz cztery interfejsy SPI, które umożliwiają komunikację z innymi urządzeniami. Jeśli chodzi o bezprzewodową komunikację, układ ESP32 posiada moduł Wi-Fi z obsługą standardu 802.11n, co umożliwia komunikację z innymi urządzeniami i sieciami w trybie bezprzewodowym, o maksymalnej prędkości do 150 Mbps. Dodatkowo, jest wyposażony w moduł Bluetooth w wersji 4.2 oraz obsługę BLE (Bluetooth Low Energy), co umożliwia bezprzewodową komunikację w krótkim zasięgu. Należy zauważyć, że układ ESP32 nie posiada wbudowanego modułu Ethernet oraz gniazda na kartę SD. Niewątpliwie jednym z kluczowych atutów układu ESP32 jest jego niskie zużycie energii, oscylujące w zakresie od 95 do 240 uA. Dzięki temu jest on idealnym wyborem do aplikacji, które wymagają oszczędności energii i działania przy niskich mocach.

Podsumowując, układ ESP32 jest bardzo wszechstronnym i uniwersalnym rozwiązaniem do budowy platform wbudowanych. Dzięki swoim parametrom, takim jak dwurdzeniowy procesor, pamięć RAM i Flash, interfejsy komunikacyjne oraz moduły Wi-Fi i Bluetooth, zapewnia on niezbędne narzędzia do stworzenia efektywnych i energooszczędnych systemów wbudowanych [1].

2.3. STM32

Płytka wyposażona została w zaawansowany mikrokontroler z rodziny STM32, oferuje wiele funkcji i możliwości, które umożliwiają precyzyjne monitorowanie oraz analizę danych. Mikrokontrolery STM32 są znane z wysokiej wydajności i elastyczności. Posiadają różne warianty, które różnią się m.in. liczbą pinów, pamięcią i mocą obliczeniową. Dzięki temu można dostosować płytę STM32 do konkretnych wymagań aplikacji.

STM32 oferuje rozbudowane interfejsy komunikacyjne, takie jak UART, SPI i I2C. Pozwala to na bezproblemowe podłączenie czujników do płytki i przesyłanie danych w sposób niezawodny. Interfejsy te umożliwiają również komunikację z innymi urządzeniami peryferyjnymi, co zwiększa funkcjonalność platformy wbudowanej. Płytka STM32 obsługuje również technologie bezprzewodowe, takie jak WiFi i Bluetooth. Dzięki temu dane mogą być przesyłane bezprzewodowo do innych urządzeń lub do chmury, co zwiększa dostępność i elastyczność systemu. Dodatkowo, istnieje wiele gotowych bibliotek i narzędzi programistycznych, które ułatwiają implementację komunikacji bezprzewodowej.

STM32 oferuje zaawansowane funkcje przerwań, które pozwalają na szybką i precyzyjną reakcję na zdarzenia wywołane przez czujniki. Można programować mikrokontroler w języku C lub C++, co daje dużą swobodę w tworzeniu aplikacji. Płytka STM32 jest również wyposażona w różne złącza i interfejsy, takie jak GPIO, ADC czy Timery, które umożliwiają bezpośrednie podłączenie czujników oraz innych modułów peryferyjnych. To zapewnia szerokie możliwości rozwoju i rozbudowy platformy wbudowanej.

Tabela 2.2: Porównanie różnych modeli STM32 z różnymi rdzeniami arm cortex M

Series STM32	L1	L4	F1	F3	F2	F4	F7
Micro	L152RE	L476RG	F103RB	F303RE	F207ZG	F401RE	F767ZI
CPU	M3	M4	M3	M4	M3	M4	M7
CPU FREQ	32Mhz	80MHz	72MHz	72MHz	120MHz	84Mhz	216MHz
RAM	80KB	128KB	20KB	64KB	128KB	96KB	512KB
FLASH	512KB	1MB	128KB	512KB	1MB	512KB	2MB
I/O	116	114	80	115	140	81	168
ADC	1x12bit 1Msps	3x12bit 5Msps	2x12bit 1Msps	1x12bit 5Msps	1x12bit 2Msps	1x12bit 2.4Msps	3x12bit 2.4Msps
Timer	1x32bit, 10x16bit	2x32bit, 14x16bit	7x16bit	1x32bit 13x16bit	2x32bit 15x16bit	2x32bit 9x16bit	2x32bit 16x16bit
DMA	1x12	1x14	1x7	1x12	1x16	1x16	1x16
USART	5	6	3	5	4	3	4

SPI	8x 16Mbit/s	3x16 Mbit/s	2x18 Mbit/s	4x16 Mbit/s	3x30 Mbit/s	4x42 Mbit/s	6x54 Mbit/s
I2C	2	3	2	3	3	3	4
Wi-Fi	None	None	None	None	None	None	None
Bluetooth	None	None	None	None	None	None	None
Ethernet	None	None	None	None	None	None	None
SD Slot	None	None	None	None	None	None	None
Zużycie energii	195 uA/MHz	100 uA/MHz	N/A	N/A	N/A	146 uA/Mhz	N/A

W tabeli 2.2 przedstawiono wybrane serie mikrokontrolerów STM32 wraz z ich parametrami. Series STM32: Tabela zawiera informacje o różnych seriach mikrokontrolerów STM32, takich jak L1, L4, F1, F3, F2, F4 i F7.

Kolumna "Micro" zawiera konkretne modele mikrokontrolerów w ramach każdej serii, takie jak L152RE, L476RG, F103RB, F303RE, F207ZG, F401RE i F767ZI. W kolumnie "CPU" znajdują się informacje o rodzaju procesora w mikrokontrolerach, takich jak M3 (Cortex-M3) i M4 (Cortex-M4). Natomiast w kolumnie "CPU FREQ" przedstawiona jest częstotliwość pracy procesora w megahercach (MHz) dla poszczególnych modeli. W kolumnie "RAM" podane są ilości pamięci RAM dostępnej w kilobajtach (KB) dla każdego modelu mikrokontrolera. Kolumna "FLASH" przedstawia pojemność pamięci flash w kilobajtach (KB) lub megabajtach (MB) dla każdego modelu. Liczba dostępnych pinów wejścia/wyjścia (I/O) w mikrokontrolerach jest przedstawiona w kolumnie "I/O". Kolumna "ADC" zawiera informacje o liczbie konwerterów analogowo-cyfrowych (ADC) oraz ich rozdzielczości w bitach dla każdego modelu. W kolumnie "Timer" przedstawiona jest liczba timerów oraz ich rozmiar (32-bitowy lub 16-bitowy) dla każdego modelu mikrokontrolera. Natomiast w kolumnie "DMA" znajduje się informacja o liczbie kanałów Direct Memory Access (DMA) dostępnych w mikrokontrolerach. Liczba interfejsów USART (Universal Synchronous/Asynchronous Receiver/Transmitter) dla każdego modelu jest przedstawiona w kolumnie "USART". Kolumna "SPI" zawiera informacje o liczbie interfejsów SPI (Serial Peripheral Interface) oraz ich maksymalnej prędkości transmisji danych

dla każdego modelu. W kolumnie "I2C" znajduje się liczba interfejsów I2C (Inter-Integrated Circuit) dla każdego modelu mikrokontrolera. Kolumny "Wi-Fi", "Bluetooth", "Ethernet" oraz "SD Slot" informują, czy dany model mikrokontrolera obsługuje odpowiednie technologie i czy posiada gniazdo na kartę SD. W kolumnie "Power need" znajdują się informacje o zapotrzebowaniu na moc w jednostkach mikroamperów na megaherc (uA/MHz) dla poszczególnych modeli.

Przedstawiona tabela dostarcza szczegółowe informacje o różnych modelach mikrokontrolerów STM32, co pozwala na wybór odpowiedniego modelu do projektu platformy wbudowanej. Dzięki swojej wydajności, elastyczności i bogactwu interfejsów komunikacyjnych, STM32 daje wiele możliwości w dziedzinie badania protokołów komunikacyjnych.

2.4. Raspberry Pi

Raspberry Pi jest popularną płytą rozwojową, która doskonale nadaje się jako podstawa do badań nad protokołami komunikacyjnymi. Płytka Raspberry Pi oferuje szerokie możliwości i wydajność, umożliwiając rozbudowane projekty związane komunikacją.

Pierwszym atutem Raspberry Pi jest jego moc obliczeniowa. Wyposażony w wydajny procesor ARM, Raspberry Pi oferuje wystarczającą moc do przetwarzania danych z czujników w czasie rzeczywistym. Procesor ten jest również odpowiedzialny za obsługę systemu operacyjnego, co umożliwia wykorzystanie bogatej gamy narzędzi i bibliotek programistycznych. Płytka Raspberry Pi posiada również bogate złącza i interfejsy, które umożliwiają łatwe podłączenie i komunikację z urządzeniami peryferyjnymi. Na płytce znajdziemy interfejsy GPIO (General Purpose Input/Output), które umożliwiają podłączenie czujników analogowych i cyfrowych. Dodatkowo, Raspberry Pi posiada złącza USB, Ethernet, Wi-Fi oraz Bluetooth, co daje możliwość łatwego podłączenia innych urządzeń i komunikację z nimi.

Raspberry Pi działa na systemie operacyjnym Linux, co zapewnia stabilność i elastyczność platformy. Można zainstalować różne narzędzia i biblioteki programistyczne umożliwiające badanie protokołów komunikacyjnych. Dodatkowo, płytka Raspberry Pi oferuje możliwość rozbudowy poprzez podłączenie modułów rozszerzeń, takich jak kamery, wyświetlacze czy karty pamięci. Daje to większe możliwości integracji i rozwinięcia funkcjonalności platformy wbudowanej.

Tabela 2.3: Porównanie modeli Raspberry Pi dostępnych na rynku

Model	Pi 1 B	Pi1 B+	Pi 2 B	Pi 3 B	Pi 4 B
CPU ARM	1176JZF-S	1176 JZF-S	Cortex-A7	Cortex-A53	Cortex-A72
CPU FREQ	700MHz	700 MHz	900Mhz	1.2GHz	1.5GHz

RAM	512MB	512 MB	1GB	1GB	2GB/4GB/ 8GB
I/O	8	17	17	17	40
ADC	Brak	Brak	Brak	Brak	Brak
UART	1	1	1	1	1
SPI	1	1	1	1	1
I2C	1	1	1	1	1
Wi-Fi	Brak	Brak	Brak	Tak	Tak
Bluetooth	Brak	Brak	Brak	Tak	Tak
Ethernet	Tak	Tak	Tak	Tak	Tak(Gigabit)
SD slot	SD	MicroSD	MicroSD	MicroSD	MicroSD
USB port	2	4	4	4	2xUSB 3.0 2xUSB 2.0
Zużycie energii	700mA	600mA	900mA	2.5A	~3A

W tabeli 2.3 przedstawiono wybrane modele komputerów jednopłytkowych serii Raspberry Pi wraz z ich parametrami. Model Pi 1 B jest wyposażony w procesor ARM 1176JZF-S działający z częstotliwością 700MHz. Posiada 512MB pamięci RAM, 8 portów I/O, 1 port UART, 1 port SPI, 1 port I2C oraz obsługuje karty pamięci typu SD. Nie posiada wbudowanego modułu Wi-Fi ani Bluetooth, ale posiada port Ethernet oraz 2 porty USB. Zużycie energii wynosi 700mA. Pi 1 B+ jest ulepszoną wersją modelu Pi 1 B. Ma identyczne parametry techniczne, ale posiada większą liczbę portów I/O, które wynosi 17. Posiada również 4 porty USB oraz obsługuje karty microSD. Zużycie energii zostało zmniejszone do 600mA. Pi 2 B jest wyposażony w bardziej wydajny procesor Cortex-A7 o częstotliwości 900MHz. Posiada 1GB pamięci RAM, 17 portów I/O, 1 port UART, 1 port SPI, 1 port I2C oraz obsługuje karty microSD. Nie posiada wbudowanego modułu Wi-Fi ani Bluetooth, ale posiada port Ethernet. Dodatkowo, model Pi 2 B posiada 4 porty USB. Zużycie energii wynosi 900mA. Pi 3 B to kolejny model z serii Raspberry Pi, który posiada jeszcze wydajniejszy procesor Cortex-A53 o częstotliwości 1.2GHz. Posiada 1GB pamięci RAM, 17 portów I/O, 1 port UART, 1 port SPI, 1 port I2C oraz obsługuje karty microSD. W przeciwieństwie do poprzednich modeli, Pi 3 B posiada wbudowany moduł Wi-Fi oraz Bluetooth. Posiada również port Ethernet i 4 porty USB. Zużycie energii wynosi 2.5A. Pi 4 B to najnowszy model z serii, wyposażony w procesor Cortex-A72 o częstotliwości 1.5GHz. W zależności od wersji, może posiadać 2GB, 4GB lub 8GB pamięci RAM. Liczba portów I/O wynosi 40. Model ten posiada 1

port UART, 1 port SPI, 1 port I2C oraz obsługuje karty microSD. Pi 4 B ma wbudowany moduł Wi-Fi oraz Bluetooth. Posiada także port Gigabit Ethernet oraz 4 porty USB (2x USB 3.0, 2x USB 2.0). Zużycie energii tego modelu wynosi około 3A.

Tabela przedstawia różne modele płyt Raspberry Pi, które oferują różne parametry techniczne i funkcjonalności. Posiadają one różne ilości pamięci RAM, portów I/O, portów komunikacyjnych (UART, SPI, I2C) oraz wsparcie dla Wi-Fi, Bluetooth, Ethernet i slotów kart pamięci. Zużycie energii również różni się w zależności od modelu. Wybór odpowiedniego modelu zależy od konkretnych potrzeb projektowych i wymagań dotyczących mocy obliczeniowej, interfejsów komunikacyjnych i dostępnych funkcji. Można wnioskować, że Raspberry Pi jest dobrą podstawą do budowy platformy wbudowanej służącej komunikacji. Dzięki swojej wydajności, interfejsom komunikacyjnym oraz elastycznemu systemowi operacyjnemu, płytka ta umożliwia rozwinięcie zaawansowanych projektów oraz łatwą implementację.

2.5. NVIDIA Jetson

Komputer Nvidia Jetson jest zaawansowaną platformą wbudowaną. Jetson to rodzina komputerów jednopłytkowych opartych na architekturze ARM, zaprojektowanych specjalnie do zastosowań wbudowanych i sztucznej inteligencji. Wśród najpopularniejszych modeli znajdują się Nvidia Jetson Nano, Jetson TX2, Jetson Xavier NX i Jetson AGX Xavier. Każdy z nich oferuje różne możliwości i moc obliczeniową, co pozwala na wybór odpowiedniego modelu do aplikacji. Komputery Nvidia Jetson są wyposażone w zaawansowane procesory graficzne Nvidia CUDA, które umożliwiają przyspieszenie obliczeń i wykorzystanie algorytmów sztucznej inteligencji.

Nvidia Jetson posiada dużą ilość portów I/O, takich jak GPIO, UART, SPI i I2C, co daje możliwość podłączanie urządzeń peryferyjnych. Można również korzystać z interfejsów Ethernet, USB i HDMI w celu łatwej komunikacji z innymi urządzeniami i wyjściem wideo.

Komputery Nvidia Jetson są wyposażone w wystarczającą ilość pamięci RAM i pamięci masowej, aby przechowywać dane z czujników i wykonywać skomplikowane obliczenia. Dostępne są różne modele z różnymi pojemnościami pamięci. Co więcej, Nvidia Jetson oferuje wsparcie dla różnych narzędzi i frameworków programistycznych, takich jak CUDA, PyTorch i TensorFlow, co ułatwia rozwój aplikacji i implementację algorytmów sztucznej inteligencji.

Tabela 2.4: Porównanie komputerów NVIDIA Jetson TK1 i TX1

	TK1	TX1
CPU ARM	A15 64-bit	A57 64-bit
CPU FREQ	933MHz	998MHz
RAM	2GB	4GB

I/O	125	70
ADC	Brak	Brak
UART	1	3
SPI	1	3
I2C	3	4
Wi-Fi	Brak	Tak
Bluetooth	Brak	Tak
Ethernet	Tak	Tak
SD slot	Tak	Tak
Zużycie energii	N/A	N/A

Tabela 2.4 przedstawia porównanie dwóch modeli komputerów Nvidia Jetson: TK1 i TX1. Model TK1 jest wyposażony w 64-bitowy procesor ARM A15 o taktowaniu 933MHz, 2GB pamięci RAM i 125 portów I/O. Nie posiada obsługi konwertera analogowo-cyfrowego (ADC), a także nie ma wbudowanej obsługi Wi-Fi, Bluetooth ani slotu na kartę SD. Jednak ma jeden port UART, jeden port SPI i trzy interfejsy I2C. Obsługuje również Ethernet. Niestety, informacja dotycząca zużycia energii nie jest dostępna. Natomiast model TX1 ma 64-bitowy procesor ARM A57 o taktowaniu 998MHz, 4GB pamięci RAM i 70 portów I/O. Podobnie jak TK1, nie posiada obsługi ADC, ale ma trzy porty UART, trzy porty SPI i cztery interfejsy I2C. Dodatkowo, ma wbudowaną obsługę Wi-Fi, Bluetooth, slot na kartę SD i obsługuje Ethernet. Informacja dotycząca zużycia energii również nie jest dostępna.

Podsumowując, model TX1 oferuje wyższą wydajność i większą ilość pamięci RAM niż TK1. Posiada również większą elastyczność dzięki większej liczbie portów UART, SPI i I2C. Dodatkowo, ma wbudowaną obsługę Wi-Fi, Bluetooth i slot na kartę SD. Oba modele obsługują Ethernet. Jednak żaden z nich nie obsługuje konwertera analogowo-cyfrowego (ADC). Wybór między TK1 a TX1 zależy od konkretnych wymagań projektowych i zastosowania platformy wbudowanej. Na podstawie powyższych można stwierdzić, że komputery Nvidia Jetson są bardzo dobrą podstawą do badań poświęconych protokołom komunikacyjnym. Oferują wysoką moc obliczeniową, bogate interfejsy komunikacyjne, wystarczającą ilość pamięci i wsparcie dla zaawansowanych narzędzi programistycznych.

2.6. Wybór platformy wbudowanej

Do realizacji badań protokołów komunikacyjnych brano pod uwagę układy ESP, STM, Raspberry Pi i Nividia Jetson. Różnią się one między sobą ceną oraz możliwościami oraz zapewniają wiele różnorodnych wbudowanych oraz dodatkowych modułów interfejsów.

Tabela 2.5: Porównanie wybranych systemów wbudowanych

	STM32F401re	Raspberry Pi 4 B	Nvidia Jetson TK1	ESP32
CPU FREQ	84Mhz	1.5 GHz	933MHz	204 Mhz
RAM	96KB	2 GB/4 GB/ 8GB	2 GB	520 KB
ADC	12-bit 2.4 Msps	Brak	Brak	8-bit 27 Ksps
Wi-Fi	Brak	Tak	Tak	Tak
Bluetooth	Brak	Tak	Brak	Tak
Ethernet	Brak	Tak	Brak	Brak
SD-slot	Brak	Tak	Tak	Brak
HDMI	Brak	Tak	Tak	Brak
Zużycie energii	146 uA/Mhz max 12 mA	~3A	N/A	95-240 uA

W procesie dobierania odpowiednich modułów, interfejsów komunikacyjnych, odpowiednich mikrokontrolerów należy wziąć pod uwagę wiele istotnych czynników, takich jak szybkość i wydajność procesora, wbudowane funkcje, możliwości rozszerzenia platformy w przyszłości. Tabela 2.5 umożliwia porównanie wszystkich rozważanych platform.

STM32F401re to mikrokontroler, który ma częstotliwość procesora wynoszącą 84 Mhz. Posiada 96 KB pamięci RAM oraz konwerter analogowo-cyfrowy (ADC) o rozdzielczości 12-bitowej i szybkości próbkowania 2.4 Msps. Nie posiada wbudowanego modułu Wi-Fi ani Bluetooth, ani portu Ethernet czy gniazda na kartę SD. Zużycie energii wynosi maksymalnie 146 uA/Mhz, a maksymalne zużycie przy pełnej pracy wynosi 12 mA.

Raspberry Pi 4B to minikomputer, który posiada wydajny procesor o częstotliwości 1.5 GHz. Jest wyposażony w 2 GB pamięci RAM oraz nie posiada wbudowanego modułu ADC. Posiada wbudowany modułu Wi-Fi, Bluetooth, oraz port Ethernet. Posiada również gniazdo na kartę SD. Raspberry Pi 4B zapewnia port HDMI, który umożliwia podłączenie do monitora lub telewizora. Zużycie energii tego minikomputera wynosi około 3 A.

Nvidia Jetson TK1 to platforma wbudowana oparta na procesorze o częstotliwości 933 MHz. Posiada 2 GB pamięci RAM oraz nie posiada wbudowanego modułu ADC. Jest wyposażony w

moduły Wi-Fi, Bluetooth oraz port Ethernet, co umożliwia komunikację bezprzewodową i połączenie z siecią. Jetson TK1 posiada również gniazdo na kartę SD oraz port HDMI. Zużycie energii dla tego układu nie jest dostępne.

ESP32 to mikrokontroler charakteryzujący się częstotliwością pracy wynoszącą 204 Mhz. Posiada 520 KB pamięci RAM oraz konwerter analogowo-cyfrowy (ADC) o rozdzielczości 8-bitowej i szybkości próbkowania 27 Ksps. Esp32 posiada wbudowane moduły Wi-Fi i Bluetooth, które umożliwiają bezprzewodową komunikację. Nie ma natomiast wbudowanego modułu Ethernet ani gniazda na kartę SD. Zużycie energii tego mikrokontrolera wynosi od 95 do 240 uA w zależności od częstotliwości.

W porównaniu tych układów, jeśli zależy nam na niskim zużyciu energii, mikrokontroler STM32F401re wyróżnia się najniższym zużyciem, wynoszącym maksymalnie 146 uA/Mhz. Jeśli potrzebujemy większej mocy obliczeniowej i większej pamięci RAM, Raspberry Pi 4B z procesorem o częstotliwości 1.5 Ghz i 2 GB pamięci RAM będzie bardziej odpowiedni. Jeśli natomiast potrzebujemy platformy wbudowanej z wbudowanym modułem Wi-Fi, Bluetooth i portem Ethernet, Nvidia Jetson TK1 oraz RaspberryPi 4B spełniają te wymagania. W przypadku ESP32, jest to mikrokontroler, który oferuje zarówno moduły Wi-Fi, Bluetooth, jak i konwerter analogowo-cyfrowy ADC, ale nie posiada modułu Ethernet. Zużycie energii tego mikrokontrolera jest również stosunkowo niskie, wynoszące od 95 do 240 uA.

Do dalszej pracy oraz realizacji części projektowej wybrałem platformę Raspberry Pi. Decyzja ta wynika przede wszystkim z dostępności sprzętu, co pozwoliło mi uniknąć dodatkowych kosztów. Dodatkowo Raspberry Pi działa na systemie Linux, który jest wygodny w obsłudze i zapewnia szerokie możliwości konfiguracji. Kolejną korzyścią jest szeroki wybór bibliotek i narzędzi, które ułatwiają badania nad różnymi protokołami komunikacyjnymi.

3. Wstęp do badania protokołów

3.1. Protokół

Protokół określa jak dane są pakowane, przesyłane i interpretowane. Na przykład protokół Modbus określa, jak struktura danych powinna wyglądać, jakie są adresy urządzeń, jak wygląda ramka danych, jakie są zasady kontroli błędów itp.

3.2. Interfejs

Interfejs określa fizyczne aspekty połączenia. Na przykład, mówi, że sygnał logiczny „1” może być reprezentowany przez napięcie od -3V do -25V a sygnał logiczny „0” przez napięcie od +3V do +25V.

3.3. Model OSI

Model OSI (Open System Interconnection) powstał na początku rozwoju sieci komputerowych. Został opracowany przez Międzynarodową Organizacje Normalizacyjną (ISO) w 1984 roku. Określa on warstwy, przez które powinna przechodzić informacja w procesie enkapsulacji. Każda warstwa ma określone funkcje i potrafi współpracować tylko z warstwą nad i pod nią [9].

Model OSI nie przyjął się w technologii, został wyparty między innymi przez TCP/IP które jest prostsze i zwięźlejsze w architekturze. Mimo że model nie znalazł zastosowania w rozwiązaniach technologicznych to nadal jest on używany w kontekście teoretycznym, gdyż warstwy przez niego opisane są wykorzystywane w komunikacji sieciowej.

Warstwy modelu OSI

1. Warstwa **Fizyczna** - odpowiada za przesyłanie bitów przez medium transmisyjne. Obejmuje fizyczne aspekty połącznia takie jak przewody, złącza, napięcia, sygnały elektryczne.
2. Warstwa **Łącza Danych** – Odpowiada za dołączenie do ramki adresów MAC (Media Access Control). Adresu MAC wbudowane są fizycznie na kartach sieciowych, są to adresy statyczne.
3. Warstwa **Sieciowa** – w tej warstwie działa protokół IPv4, zajmuje się on dołączaniem do ramek adresów IP. Warstwa zarządza trasowaniem danych między różnymi sieciami, oraz za fragmentacje danych.
4. Warstwa **Transportowa** – ta warstwa obsługuje nawiązywanie i utrzymywanie połączeń ,tak zwanych sesji, pomiędzy dwoma urządzeniami końcowymi. Warstwa obsługuje również niezawodność komunikacji. W tej warstwie działają protokoły TCP oraz UDP.
5. Warstwa **Sesji** – warstwa teoretyczna, rzadko znajduje zastosowanie w rzeczywistych protokołach, jej celem miało być zarządzanie sesjami komunikacyjnymi między aplikacjami oraz ich synchronizacja.
6. Warstwa **Prezentacj**i – jej zadaniem jest tłumaczenie danych między formatem używanym przez aplikacje a formatem obsługiwanym przez sieć.
7. Warstwa **Aplikacji** – warstwa, do której dostęp ma użytkownik, obsługuje protokoły takie jak HTTP czy FTP.

4. Protokoły komunikacyjne – projekty z implementacją

4.1. Protokoły sieciowe

4.1.1. TCP

- *Cel i zastosowanie protokołu*

Protokół TCP (Transmission Cotrol Protocol) to siła napędowa internetu. Zaprojektowany został w celu zapewnienia niezawodnej transmisji strumieni bajtowych na bazie międzysieci. Protokół TCP zaprojektowany jest tak aby mógł radzić sobie z lokalnymi różnicami pomiędzy sieciami.

- *Opis działania protokołu*

Usługa TCP bazuje na punktach dostępowych – Gniazdach, tworzonych zarówno przez odbiorcę i nadawcę. Każde gniazdo jest w istocie konkatenacją adresu IP hosta oraz lokalnego dla tego hosta 16-bitowego numeru zwanego portem. Połączenia TCP posiadają możliwość przesyłania danych w obu kierunkach, są one połączeniami dwupunktowymi [2]. TCP jest protokołem działającym w trybie klient-serwer. TCP gwarantuje wyższym warstwom komunikacyjnym

dostarczenie wszystkich pakietów w całości, z zachowaniem kolejności i bez duplikatów. Zapewnia to wiarygodne połączenie kosztem większego nagłówka oraz większej ilości wysyłanych pakietów.

W protokole TCP do nawiązania połączenia używa się procedury three-way handshake. Procedura służy do synchronizacji wirtualnego połączenia [2].

W pierwszym kroku host inicjujący (klient) wysyła segment z ustawionym bitem SYN do serwera, W drugim kroku serwer, który otrzymał segment z ustawionym bitem SYN, odpowiada segmentem, w którym są ustawione bity SYN i ACK. W ostatnim trzecim kroku klient odsyła do serwera paczkę potwierdzającą że otrzymał segment z bitami SYN i ACK do serwera. W tym momencie zostaje nawiązane połączenie.

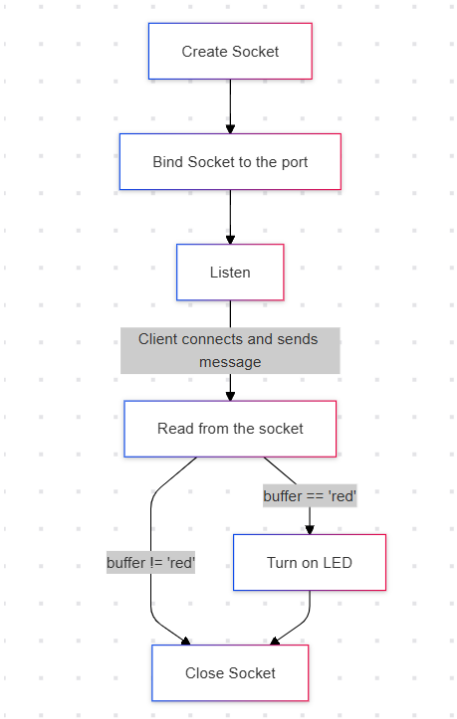
Do zbudowania połączenia TCP, potrzebne są gniazda (z ang. sockets). Gniazda są kanałami komunikacyjnymi, które umożliwiają wymianę danych pomiędzy niepowiązanymi procesami lokalnie i w sieciach. Pojedyncze gniazdo jest jednym punktem końcowym dwukierunkowego kanału komunikacyjnego.

Model TCP/IP wyparł teoretyczny Model OSI i stał się praktycznym modelem warstwowym wykorzystywanym w sieciach komputerowych. Łączy on protokoły TCP i IP. Protokół TCP zapewnia niezawodność Transmisji natomiast IP zajmuje się adresowaniem.

Protokół IP to podstawowy protokół warstwy sieciowej modelu OSI stosowany w sieciach komputerowych. Do jego funkcji należy adresowanie urządzeń komunikujących się adresem IP, oraz fragmentacja danych na mniejsze paczki [2]. Protokół IP jest protokołem zawodnym, nie gwarantuje czy pakiety dotrą do adresata, czy dotrą w tej samej kolejności w jakiej zostały wysłane lub czy nie zostaną zdublowane. Dlatego najczęściej protokół IP jest wykorzystywany w stosie TCP/IP.

- *Opis stworzonego projektu*

Projekt wykorzystujący protokół TCP, polegał na napisaniu dwóch programów: serwera i klienta. Serwer uruchamiany był na Raspberry Pi, natomiast klient na laptopie, konkretnie na wirtualnej maszynie z oprogramowaniem Linuxa Ubuntu. Oba urządzenia znajdowały się w tej samej sieci lokalnej WLAN. Pierwszy projekt polega na sterowaniu z poziomu klienta wyjściem malinki. Na polecenie klienta (wysłanie stringa „red”) zostaje załączona czerwona dioda połączona z wyjściem malinki.



Rysunek 4.1 Flowchart działania programu

- *Kod źródłowy, fragmenty z objaśnieniami*

Server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
void error(const char *msg) {
perror(msg); // print error msg
exit(1); // exit the main() function }

int main(int argc, char *argv[])
{
int sockfd, newsockfd, portno; // defines ints
socklen_t clilen; //size of address
char buffer[256]; // buffer
struct sockaddr_in serv_addr, cli_addr; // defines structures
int n;
if (argc < 2) {
fprintf(stderr,"ERROR, no port provided\n"); // deal with wrong port
exit(1);
}
```

```
sockfd = socket(AF_INET, SOCK_STREAM, 0); // makes a socket
if (sockfd < 0)
error("ERROR opening socket"); // if socket creation failed
bzero((char *) &serv_addr, sizeof(serv_addr));
portno = atoi(argv[1]); // get port number
serv_addr.sin_family = AF_INET; // specify address family (IPv4)
serv_addr.sin_addr.s_addr = INADDR_ANY; // bind IP addresses
serv_addr.sin_port = htons(portno); // set port number
if (bind(sockfd, (struct sockaddr *) &serv_addr,
sizeof(serv_addr)) < 0)
error("ERROR on binding"); // error while binding socket
listen(sockfd,5); // listen on created socket
clilen = sizeof(cli_addr);
newsockfd = accept(sockfd,
(struct sockaddr *) &cli_addr,
&clilen); // creates new socket for first incoming connection
if (newsockfd < 0)
error("ERROR on accept"); // error while creating socket
bzero(buffer,256);
n = read(newsockfd,buffer,255); // read from connection socket
if (n < 0) error("ERROR reading from socket"); // if error
printf("Here is the message: %s\n",buffer); // show the message

      // Remove newline character if present
    buffer[strcspn(buffer, "\r\n")] = 0;

    // Check if the buffer contains the string "red"
    if (strcmp(buffer, "red") == 0) {
        // Call the Python script to turn on the LED
        if (system("python3 /home/student/led_on.py > /home/stu-
dent/led_on_output.log 2>&1") == -1) {
            perror("Error executing system command");
        } else {
            printf("Warunek spełniony\n");
        }
    } else {
        printf("Warunek nie jest spełniony\n");
    }

n = write(newsockfd,"I got your message",18);

if (n < 0) error("ERROR writing to socket"); //
close(newsockfd); // close connection socket
close(sockfd); // close listening socket
return 0;
}
```

Podczas tworzenia gniazda metodą socket definiujemy trzy parametry:

```
int socket(int domain, int type, int protocol);
```

Domain(domena lub rodzina protokołu), określa rodzinę protokołów, której będzie używać gniazdo. Najczęściej spotykane to:

- „AF_INET”: adresy IPv4 (Internet Protocol v4)
- „AF_INET6”: adresy IPv6 (Internet Protocol v6)
- „AF_UNIX” lub „AF_LOACL”: Lokalne gniazda unixowe (służą tylko do komunikacji między procesami na tym samym komputerze)

Type(typ gniazda), określa semantykę komunikacji, czyli sposób, w jaki dane będą przesyłane. Najczęściej używane wartości:

- „SOCK_STREAM”: Gniazdo strumieniowe, używane w protokole TCP (Transmission Control Protocol). Zapewnia niezawodny, uporządkowany i dwukierunkowy strumień bajtów.
- „SOCK_DGRAM”: Gniazdo datagramowe, używane w protokole UDP(User Datagram Protocol). Umożliwia wysyłanie niezawodnych nieuporządkowanych i bezpołączeniowych datagramów.
- „SOCK_RAW”: Gniazdo surowe, pozwala na dostęp do podstawowych protokołów sieciowych, takich jak ICMP(Internet Control Message Protocol). Umożliwia bezpośrednią komunikację z warstwą IP.

Protocol(protokół), precyzuje jaki konkretny protokół chcemy użyć z danej rodziny protokołów. Ustawiony na „0” oznacza, że będzie wybrany domyślny protokół dla zdefiniowanej w pierwszym i drugim argumencie rodziny. Przykłady:

- „IPPORTO_TCP”: Protokół TCP
- „IPPORTO_UDP”: Protokół UDP
- „IPPORTO_ICMP”: Protokół ICMP

- *Podsumowanie wyników działania*

Z wykorzystaniem stos protokołów TCP/IP został zaimplementowany prosty system klient-serwer. Serwer uruchamiany na RaspberryPi odbiera dane wysyłane przez klienta z komputera i sterują podłączoną diodą LED. Projekt pozwala na praktyczną demonstrację działania protokołów TCP/IP , która może być wykorzystywana w systemach wymiany danych w IoT.

4.1.2. FTP

- *Cel i zastosowanie protokołu*

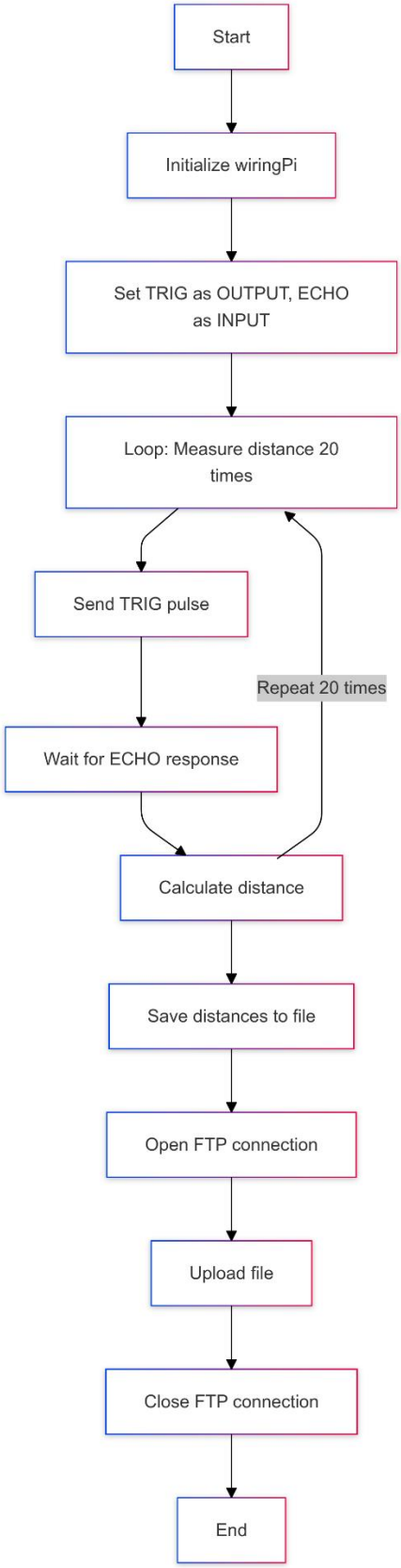
FTP (File Transfer Protocol) to popularny protokół warstwy aplikacji służący do wysyłania plików pomiędzy urządzeniami znajdującymi się w sieci.

- *Opis działania protokołu*

FTP wykorzystuje w niższej warstwie TCP. Protokół ten można wykorzystać do wysyłania plików z jednego urządzenia do drugiego pełniącego rolę serwera, posiada możliwość autoryzacji użytkownika i hasło. FTP nie jest szyfrowany, istnieją alternatywy takie jak SFTP (SSH File Transfer Protocol) które zapewniają bezpieczne, szyfrowane przesyłanie danych.

- *Opis stworzonego projektu*

Projekt zrealizowano wykorzystując RaspberryPi, czujnik odległości hc-sr04, laptop z oprogramowaniem Windows. System Windows posiada wbudowaną możliwość uruchomienia serwera FTP, używa się do tego narzędzia IIS (Internet Information Services). Tworzymy taki serwer, oraz przypisujemy mu użytkownika i hasło. Kolejnym krokiem jest napisanie programu na raspberryPi, który za pomocą czujnika zbiera określoną ilość danych, zapisuje dane do pliku tekstowego a następnie wysyła dany plik na serwer FTP. Aby serwer mógł przyjąć pliki pochodzące „z zewnątrz” trzeba wcześniej jeszcze wyłączyć funkcję Windows Defender, która chroni Windowsa. W naszym przypadku, wystarczy wyłączyć ochronę w sieci lokalnej, gdyż RaspberryPi jest podłączone do tego samego routera WiFi.



Rysunek 4.2 Flowchart działania programu

- *Kod źródłowy*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <wiringPi.h>
#include <curl/curl.h>

#define TRIG 4  // GPIO 23 (wiringPi pin 4)
#define ECHO 5  // GPIO 24 (wiringPi pin 5)

void measure_distance(float *distances, int num_measurements) {
    int i;
    for (i = 0; i < num_measurements; i++) {
        digitalWrite(TRIG, LOW);
        usleep(2000000);  // sleep for 2 seconds

        digitalWrite(TRIG, HIGH);
        usleep(10);  // sleep for 10 microseconds
        digitalWrite(TRIG, LOW);

        while (digitalRead(ECHO) == LOW);
        long startTime = micros();

        while (digitalRead(ECHO) == HIGH);
        long travelTime = micros() - startTime;

        float distance = travelTime / 58.0;
        distances[i] = distance;
        printf("Distance: %.2f cm\n", distance);
    }
}

void write_to_file(float *distances, int num_measurements, const char
*file_name) {
    FILE *file = fopen(file_name, "w");
    if (file == NULL) {
        perror("Failed to open file");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < num_measurements; i++) {
        fprintf(file, "%.2f cm\n", distances[i]);
    }

    fclose(file);
}
```

```c
void upload_file_to_ftp(const char *file_name, const char *ftp_url, const char
*ftp_user, const char *ftp_password) {
    CURL *curl;
    CURLcode res;
    FILE *file;

    curl_global_init(CURL_GLOBAL_DEFAULT);
    curl = curl_easy_init();

    if (curl) {
        file = fopen(file_name, "rb");
        if (!file) {
            perror("Failed to open file for reading");
            curl_easy_cleanup(curl);
            curl_global_cleanup();
            exit(EXIT_FAILURE);
        }

        curl_easy_setopt(curl, CURLOPT_UPLOAD, 1L);
        curl_easy_setopt(curl, CURLOPT_URL, ftp_url);
        curl_easy_setopt(curl, CURLOPT_USERPWD, ftp_user);
        curl_easy_setopt(curl, CURLOPT_PASSWORD, ftp_password);
        curl_easy_setopt(curl, CURLOPT_READDATA, file);

        res = curl_easy_perform(curl);
        if (res != CURLE_OK) {
            fprintf(stderr, "curl_easy_perform() failed: %s\n",
curl_easy_strerror(res));
        }

        fclose(file);
        curl_easy_cleanup(curl);
    }

    curl_global_cleanup();
}
```

```
int main() {
    if (wiringPiSetup() == -1) {
        perror("wiringPiSetup failed");
        return 1;
    }

    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);

    int num_measurements = 20;
    float distances[num_measurements];

    measure_distance(distances, num_measurements);

    const char *file_name = "distance_measurements.txt";
    write_to_file(distances, num_measurements, file_name);

    const char *ftp_url = "ftp://192.168.100.45/distance_measurements.txt";
    const char *ftp_user = "ftp-user";
    const char *ftp_password = "wimirAGH24";

    upload_file_to_ftp(file_name, ftp_url, ftp_user, ftp_password);

    return 0;
}
```

- *Podsumowanie wyników działania*

Projekt został zrealizowany z wykorzystaniem RaspberryPi, czajnika odległości HC-SR04 oraz serwera FTP na systemie Windows. Raspberry zbiera dane w wyjścia czujnika, zapisuje je do pliku a następnie wysyła na serwer FTP. Projekt pokazuje jak w prosty sposób można wykorzystać protokół FTP do wysyłania plików w sieci lokalnej.

4.1.3. MQTT

- *Cel i zastosowanie protokołu*

MQTT (TU BRAKUJE) to niezwykle lekki i prosty protokół nadający się idealnie do użytku w aplikacjach z ograniczoną przepustowością takie jak komunikacja maszyna-maszyna czy IoT. Co ciekawe z protokołu MQTT korzysta Facebook Messenger.

- *Opis działania protokołu*

MQTT to protokół oparty o wzorzec publikacja/subskrypcja. Protokół został zbudowany na bazie TCP. Do komunikacji potrzebny jest Broker, pełniący rolę serwera oraz klient. Klient ma dwie możliwości, może publikować temat lub subskrybować temat. Klientów może być wielu. Kiedy klient zasubskrybuje dany temat, będzie otrzymywać wszystkie informacje publikowane

pod tym tematem. Dla lepszej organizacji tematy na brokerze są strukturyzowane tak jak foldery i pliki na popularnych Systemach operacyjnych takich jak Linux czy Windows. Dla przykładu załóżmy, że temat to HalaB/MaszynaX/temperatura, każdy klient który zasubskrybuje ten temat będzie dostawać informacje na temat temperatury, która może być wysyłana do brokera z czujnika zamontowanego na maszynie.

- *Opis stworzonego projektu*

Do praktycznego wykorzystania protokołu MQTT potrzebna jest biblioteka mosquitto. Zapewnia ona łatwą implementację programów publikujących oraz subskrybujących. Broker działa na urządzeniu jako deamon (deamon - proces działający na urządzeniu domyślnie „w tle” od razu po włączeniu urządzenia).

Rysunek 4.3 Flowchart działania programu

- *Kod źródłowy, fragmenty z objaśnieniami*

```c
#include <stdio.h>
#include <mosquitto.h>

int main() {
    int rc;
    struct mosquitto * mosq;

    mosquitto_lib_init();
    mosq = mosquitto_new("publisher-test", true, NULL);
    rc = mosquitto_connect(mosq, "localhost", 1883, 60);
    if(rc != 0) {
        printf("Client could not connect to broker! Error Code: %d\n", rc);
        mosquitto_destroy(mosq);
        return -1; }

    printf("We are now connected to the broker!\n");
    mosquitto_publish(mosq, NULL, "test/t1", 6, "Hello", 0, false);
    mosquitto_disconnect(mosq);
    mosquitto_destroy(mosq);
    mosquitto_lib_cleanup();
    return 0;
}
```

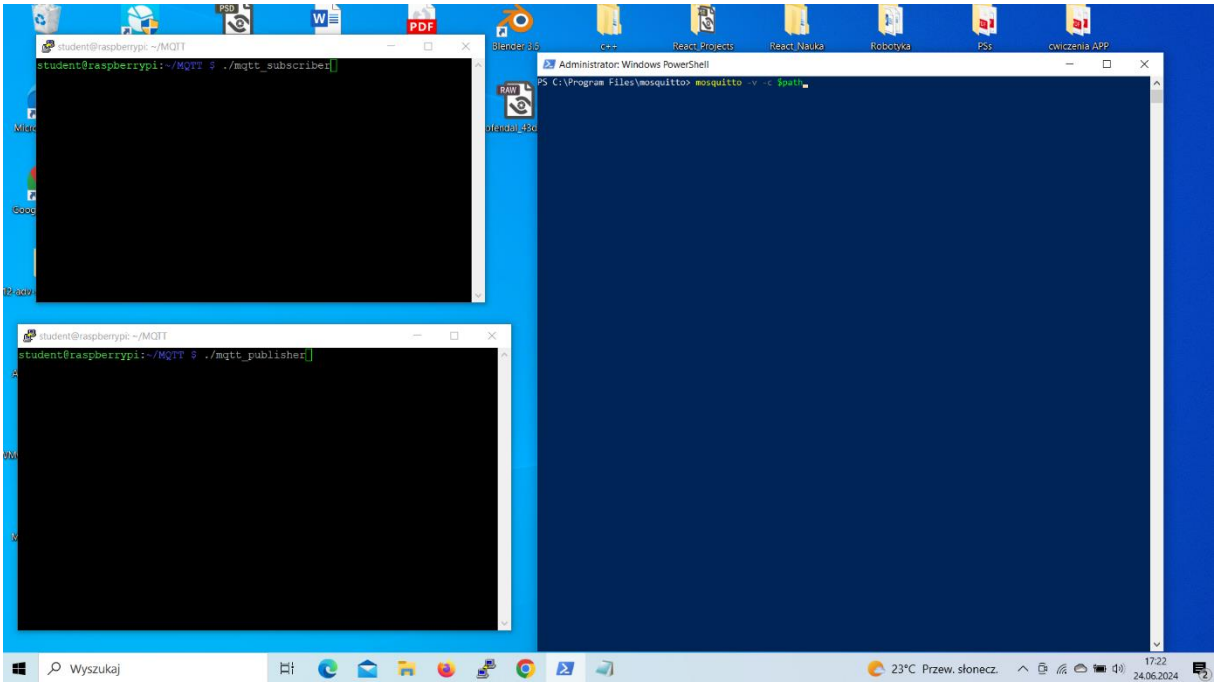
Za pomocą metody mosquitto_new tworzymy nowego klienta który jest wskaźnikiem na strukturę mosquitto. Pierwszy argument to ID klienta, drugi argument informuje nas o tym czy chcemy użyć clean session, flaga ta określa czy broker powinien przechowywać informacje o sesji klienta po jego rozłączeniu, trzeci argument pozwala na przekazanie loginu i hasła jeżeli używamy autentyfikacji na brokerze.

Za pomocą metody mosquitto_publish publikujemy wiadomość pod danym tematem. Pierwszy argument wskazuje na strukturę reprezentującą klienta, drugi argument to nr ID wiadomości, my ustawiamy na NULL, następnie przekazany jest temat pod którym chcemy publikować na Brokerze, liczba bajtów naszej wiadomości, oraz wiadomość. Ostatnie dwa argumenty to quality od service oraz flaga retain określająca czy wiadomość ma zostać zachowana na brokerze.

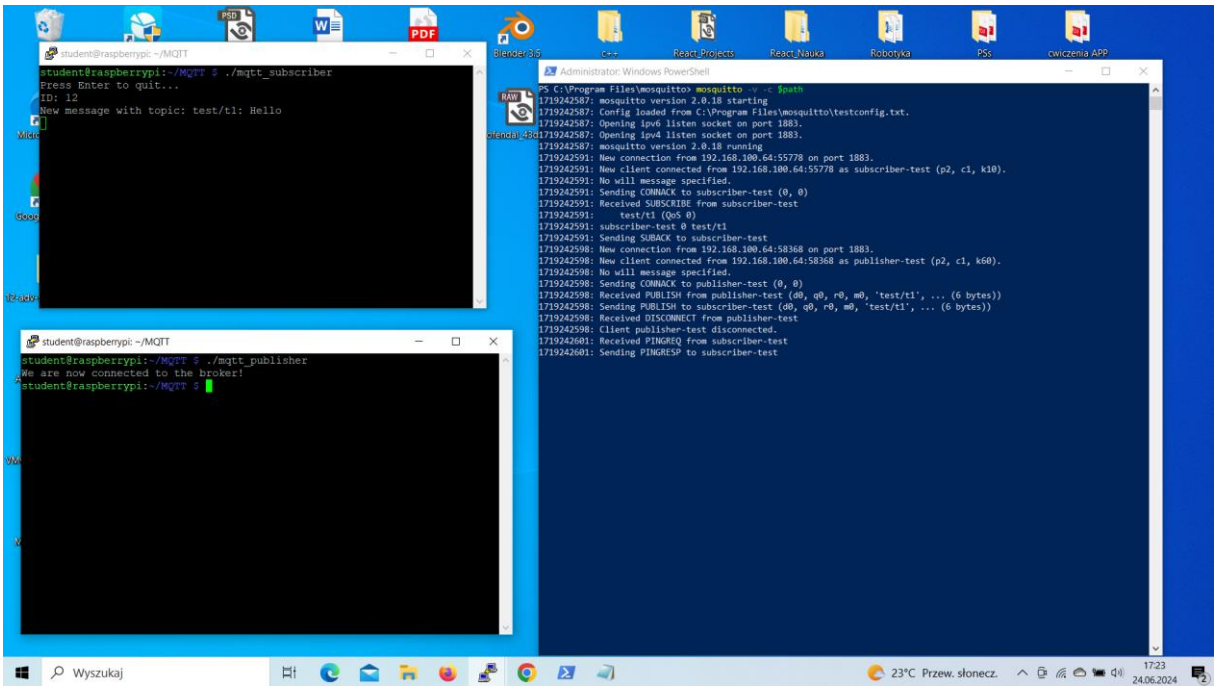
- *Podsumowanie wyników działania*

Powyższy kod można testować na localhost'cie, ciekawsze jednak będzie rozwiązanie, w którym broker i klient będą dwoma różnymi urządzeniami znajdującymi się w sieci. Do realizacji takiego projektu zainstalowano paczkę mosquitto na laptopie z Windowsem 10 a program mqtt_publisher, oraz mqtt_subscriber umieszczono, skompilowano i uruchomiono na RaspberryPi. Projekt działa w sieci LAN. Najpierw na Windowsie należy odpalić Brokera, w katalogu gdzie zainstalowane zostało mosquitto należy dodać plik konfiguracyjny zezwalający na połączenia z zewnątrz. Konieczne również jest wyłączenie Windows Defendera dla sieci lokalnej. Ostatnim krokiem jest zmiana argumentu w metodzie mosquitto_connect, która odpowiada za połączenie klienta z brokerem. „Localhost" należy zmienić na adres Brokera w

sieci LAN. Działanie programu przedstawiono na zdjęciach poniżej. W pracy nie zamieszczono kodu subscribera, jest podobny do programu publishera i nie ma w nim rzeczy dotychczas nie wspomnianych.



Rysunek 4.4 Publisher oraz Subscriber uruchomione na RaspberryPi, broker uruchomiony na Windowsie.



Rysunek 4.5 Wynik działania aplikacji.

4.1.4. Modbus TCP

- *Cel i zastosowanie protokołu*

Modbus został opracowany w 1979 roku przez firmę Modicon. Został zaprojektowany specjalnie do użytku w ówczesnych sterownikach PLC firmy Modicon. Dziś jest to otwarty protokół to znaczy, że każda firma może implementować ten protokół w swoich urządzeniach, co daje mu dużą uniwersalność. Istnieją trzy główne rodzaje protokołu Modbus: ASCII, RTU oraz TCP/IP [5]. Co ważne systemy te nie komunikują się między sobą

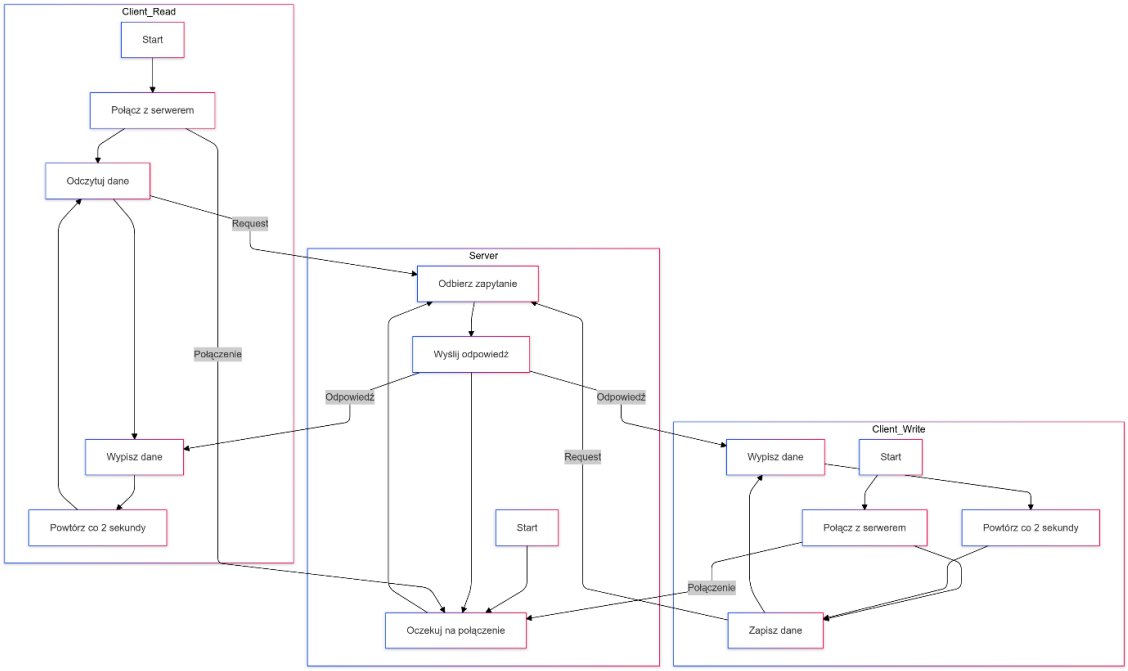
- *Opis działania protokołu*

W protokole modbus zdefiniowano dwa rodzaje danych: **cewki** oraz **rejestry**. Cewki reprezentują pojedyncze bity. Zakres adresowy 1 -9999 przypisano cewkom, służącym jedynie do odczytu. Cewki z zakresu 10001-19999 możemy zapisywać i odczytywać (te adresy mają też nazwę **Descrete inputs**). Rejestry to jednostki danych o szerokości 16 bitów. Każdy rejestr może przechowywać liczby całkowite w zakresie od 0 do 65535 (00h – FFFFh). Rejestry spod adresów 30001-39999 służą tylko do odczytu są to tzw. **Rejestry wejściowe.** Rejestry spod adresów 40001 -49999 można odczytywać oraz modyfikować, nazywamy je **holding registers**.

W Przeciwieństwie do wersji RTU Modbus TCP korzysta z łącza Ethernet. Różni się także modelem komunikacji. Zamiast Master-Slave mamy tu do czynienia z Serwer-Client, co daje możliwość na komunikacje serwera w jednym momencie z wieloma klientami. Różni się też paczka wysłanej informacji, do kodu funkcji oraz samych danych zostaje w Modbusie TCP dołączony nagłówek MBAP, służący do identyfikacji wiadomości.

- *Opis stworzonego projektu*

W ramach badań nad protokołem MODBUS stworzono 3 programy w języku c umożliwiające komunikacje międzyprocesową na urządzeniu raspberryPi. Zaprogramowano klienta zapisującego dane, klienta odczytującego dane, oraz serwer.



Rysunek 4.6 Flowchart działania programu

- *Kod źródłowy, fragmenty z objaśnieniami*

Modbus_client_read.c

```
#include <stdio.h>
#include <modbus/modbus.h>
#include <errno.h>
#include <unistd.h>

int main(void)
{
    modbus_t *ctx;
    int rc = 0;
    int i;
    uint16_t tab_reg[64];

    ctx = modbus_new_tcp("192.168.100.64", 1502);

    while (modbus_connect(ctx) == -1)
    {
        fprintf(stderr, "Connection failed: %s\n", modbus_strerror(errno));
    }

}
    while(1)
    {
        rc= modbus_read_input_registers(ctx, 0, 10, tab_reg);
        if(rc == -1) {
            fprintf(stderr, "%s\n", modbus_strerror(errno));
            break;
        }

        for (i=0; i < rc; i++) {
            printf("reg[%d]=%d (0x%X)\n", i, tab_reg[i], tab_reg[i] );
        }
        sleep(2);
    }

    printf("freeing up resourses\n");
    modbus_close(ctx);
    modbus_free(ctx);
    return 0;
}
```

Pierwszym krokiem jest zainstalowanie biblioteki libmodbus z repozytorium github na nasze raspberryPi. Pisanie kodu zaczynamy od zdefiniowania paru niezbędnych zmiennych, struktury modbus_t ze wskaźnikiem na zmienną context, pomocniczą zmienną typu int rc która będzie zwracać wynik odpowiedzi operacji read lub write. Na końcu definujemy buffer w którym przechowamy dane odczytane ze serwera. Funkcja modbus_new_tcp tworzy nowy context tcp, w argumentach przekazujemy adres urządzenia pełniącego rolę serwera oraz port na którym odbywać się będzie komunikacja, domyślnie dla komunikacji modbus jest to port nr 502, my jednak żeby używać portów wolnych korzystamy z portu nr 1502. Metoda modbus_connect służy do nawiązania połączenia. Jeśli funkcja ta zwraca wynik -1 to znaczy, że połączenie się nie udało. Jeżeli udało się nawiązać połączenie, wchodzimy do nieskończonej pętli while(1), w której metodą modbus_read_input_registers odczytujemy rejestry. Funkcja modbus_read_input_registers bierze 4 argumenty, pierwszy to context, struktura ta zawiera informacje o połączeniu modbus, następnie adres rejestru początkowego, liczba rejestrów do odczytu, i na koniec tablica (wskaźnik) do bufora, w którym zostaną zapisane wartości rejestrów. Funkcja modbus_read_input_registers zwraca liczbę rejestrów które przeczytała. Jeżeli nie przeczytała nic to zwraca wynik -1. Na koniec należy zamknąć połączenie metodami modbus_close() oraz modbus_free() które przyjmują wskaźnik na strukturę context.

W programie służącym do zapisu danych zmieniam metodę read na metodę write. To jedyna różnica dlatego nie zamieszczam w pracy kodu tego programu.

modbus_serwer.c

```c
#include <stdio.h>
#include <modbus/modbus.h>
#include <errno.h>
#include <unistd.h>

const int NB_BITS = 16;
const int NB_INPUT_BITS = 16;
const int NB_REGISTERS = 16;
const int NB_INPUT_REGISTERS = 16;

const uint8_t UT_INPUT_BITS_TAB[] = {0xFF, 0xDB};

int main(void){
int s = -1;
int rc = 0;
int i;
uint8_t buf[MODBUS_TCP_MAX_ADU_LENGTH] = {};
modbus_t *ctx;
modbus_mapping_t *map;

ctx = modbus_new_tcp(NULL, 1502);

map =  modbus_mapping_new_tart_address(0, NB_BITS, 0, NB_INPUT_BITS, 0, NB_RE-
GISTERS, 0, NB_INPUT_REGISTERS);

if(map == NULL) {
    fprintf(stderr, "Failed to allocate the mapping: %s\n", modbus_strer-
ror(errno));
    return -1;
}
else {
    printf("mapping created\n");
    printf("Size od holding registers is %ld \n", sizeof(map->tab_input_regi-
sters));  }
}
```

```
map->tab_input_registers[0] = 20;
map->tab_input_registers[1] = 22;
map->tab_input_registers[2] = 222;

modbus_set_bits_from_bytes(
    map->tab_input_bits, 0, NB_INPUT_REGISTERS, UT_INPUT_BITS_TAB);

for(i=0; i < NB_INPUT_REGISTERS; i++) {
    printf("Discrete Input: %d\n", map->tab_input_bits[i]);


while(1) {
    s = modbus_tcp_listen(ctx, 1);
    modbus_tcp_accept(ctx, &s);

    if(s < 0) {
      printf("Error listening for modbus, err %s\n", modbus_strerror(errno));
      usleep(100000);
      continue;
    } else  {
        break;
    }
}

struct mb_tcp {
    uint16_t transact;
    uint16_t protocol;
    uint16_t length;
    uint8_t unit;
    uint8_t func;
    uint8_t data[];
}

for(;;){
    do{
        rc = modbus_receive(ctx, buf);
    }while (rc == 0);

    if(rc < 0) {
        printf("%d\n", rc);
        printf("Error %s\n", modbus_strerror(errno));
        close(s);
        break;
    }
```

```
    printf("Recived poll from Master\n");
    struct mb_tcp *frame = (struct mb_tcp *)buf;
    printf("Function code is: %d\n", frame->func);
    printf("Protocol is: %d\n", frame->protocol);
    printf("Transaction is: %d\n", frame->transact);
    printf("-------------------------------\n");

    rc = modbus_reply(ctx, buf, rc, map);
    if (rc < 0) {
        printf("%d%d\n", rc, rc);
        printf("Error %s\n", modbus_strerror(errno));
        close(s);
        break;
    }
}

close(s);
modbus_free(ctx);
modbus_mapping_free(map);

return 0;
}
```

Skupmy się na poleceniach, których nie było w programie klienta. Po pierwsze tworzymy zmienną s, która będzie potrzebna podczas tworzenia gniazda (**socket**). Kolejną nową zmienną którą musimy zdefiniować jest wskaźnik na strukturę **modbus_mapping_t.** W funkcji modbus_new_tcp nie podajemy adresu klienta, chcemy aby połączyć się mógł dowolny klient znający nasz adres. Standardowo jednak definiujemy port, taki sam jak na kliencie, którego napisaliśmy, bez tego nasze procesy nie nawiążą połączenia. W funkcji **modbus_mapping_new_start_address()** inicjalizujemy mapowanie odpowiednimi ilościami cewek, wejść dyskretnych, rejestrów trzymających i rejestrów wyjściowych. Za pomocą funkcji **modbus_set_bits_from_bytes()** możemy obsłużyć wejścia dyskretne, to znaczy że możemy przekonwertować zmienną o wielkości bajta na bity. Funkcja **modbus_tcp_listen** oraz **modbus_tcp_accept** definiują nasłuchiwanie na połączenie TCP na gnieździe. **Modbus_recive** nasłuchuje na połączenia, i przypisuje dane do buffera gdy nawiązane zostanie połączenie. Tworzymy strukturę ramki modbus **mb_tcp**. Rzutujemy otrzymane dane z buffera na nową zmienną typu **mb_tcp**. Na koniec odsyłamy otrzymaną informację do mastera, służy do tego funkcja **modbus_reply()**. Po zakończeniu programu zamykamy gniazdo (socket) oraz czyścimy pamięć metodami **modbus_free()**, **modbus_mapping_free()**.

- *Podsumowanie wyników działania*

Projekt pokazuje praktyczne działanie protokołu Modbus TCP, dane muszą być zdefiniowane w określony sposób, typowy dla sterowników PLC jednak nie typowy dla RaspberryPi. Projekt ma

charakter demonstracyjny i służy do zgłębienia wiedzy na temat protokołu. Podstawową różnicą między Modbus TCP a RTU jest wykorzystywane medium transmisyjne, Ethernet, oraz różna architektura: Dla Modbus TCP mamy klient-serwer a dla Modbus RTU analogicznie master-slave. Klient pełni role mastera ,który odpytuje oraz inicjuje połączenie. Serwer, tak jak slave, jedynie udostępnia dane.

4.1.5. HTTP

- *Cel i zastosowanie protokołu*

Żądanie http jest kompletnym pakietem informacji, który jest przesyłany z klienta do serwera w celu wykonania określonej operacji na zasobie serwera. Protokół standaryzuje komunikacje w internecie dzięki czemu różne systemy informatyczne mogą ze sobą efektywnie współpracować [10]. HTTP to protokół tekstowy, nie binarny, dlatego znajduje zastosowanie np. w internecie, ale nie w IoT.

- *Opis działania protokołu*

Projekt polega na wykorzystaniu raspberryPi jako serwera www/http. Na bazie tego serwera została zbudowana aplikacja, formularz kontaktowy, umożliwiająca wysyłanie wiadomości mailowych z poziomu strony internetowej.

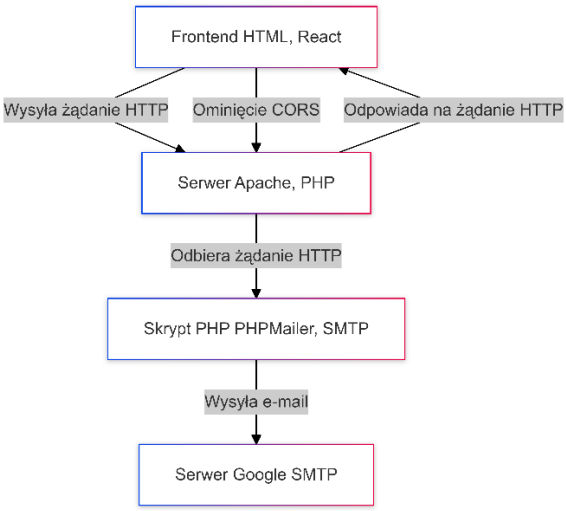
- *Opis stworzonego projektu*

Pierwszym zadaniem jest stworzenie na urządzeniu pełniącym funkcje serwera środowiska LAMP (Linux, Apache, MySql, Php), które oznacza zestaw oprogramowania potrzebny do utworzenia platformy serwerowej utrzymującej dynamiczne strony www. Na początek na systemie należy zainstalować Apache, następnie interpreter języka PHP, który służy do programowania skryptów wykonujących się po stronie serwera.

Po zainstalowaniu powyższych pakietów oraz wpisaniu w pasek adresu numeru IP raspberryPi zobaczymy dokumenty html, które umieściliśmy na serwerze. Umieszczanie plików na serwerze sprawdza się do przeniesienia plików do odpowiedniego folderu zainstalowanego wraz z paczką Apache np. za pomocą programu WinSCP.

W katalogu będącym zasobem serwera umieszczamy skrypt php służący do wysyłania maili, korzystamy z biblioteki PHPMailer oraz serwerów Google'a, korzystających z protokołu SMTP (Simple Mail Transfer Protocol), czyli popularnej poczty gmail. Jest to backend naszej aplikacji. Frontend zostaje zbudowany na językach do programowania aplikacji webowych czyli html oraz react (framework języka javascript). Przesył danych pomiędzy tymi dwoma „końcami" czyli frontendem uruchomionym w przeglądarce klienta a serwerem działającym na raspberryPi odbywa się za pomocą żądania HTTP (HyperText Transfer Protocol). Żądanie http jest kompletnym pakietem informacji, który jest przesyłany z klienta do serwera w celu wykonania określonej operacji na zasobie serwera. Protokół standaryzuje komunikacje w internecie dzięki czemu różne systemy informatyczne mogą ze sobą efektywnie współpracować. HTTP to protokół tekstowy, nie binarny, dlatego znajduje zastosowanie np. w internecie, ale nie w IoT. Przeglądarka wysyła żądanie http, natomiast serwer, na którym uruchomiony jest skrypt PHP nasłuchuje na określonym porcie i odbiera żądanie z pakietem danych, następnie sesja jest

zamykana. Po drodze należy jeszcze ominąć CORS (Cross-Origin Resource Sharing) czyli mechanizm bezpieczeństwa kontrolujący dostęp do zasobów pomiędzy domenami.



Rysunek 4.7 Flowchart działania programu

- *Kod źródłowy, fragmenty z objaśnieniami*

Klient który uruchomiony zostaje w przeglądarce internetowej wysyła żądanie http do serwera za pomocą javascript'owej metody fetch. Słowo kluczowe await związane jest z asynchronicznąścią metody fetch. Kod po słowie kluczowym await jest wykonywany asynchronicznie, to znaczy nie blokuje reszty programu. Metoda POST oznacza, że będziemy umieszczać dane na serwerze. Nagłówki informują serwer o tym jakiego typu danych ma się spodziewać. Sekcja Body to dane wysyłane do serwera

```
const response = await fetch(
        "http://192.168.100.64/forbot/verify_recaptcha.php",
        {
          method: "POST",
          headers: {
            "Content-Type": "application/x-www-form-urlencoded",
          },
          body: `captchaValue=${captchaValue}`,
        }
```

Funkcja wysyłające dane do weryfikacji reCaptcha.

```
fetch("http://192.168.100.64/forbot/save.php", {
      method: "POST",
      body: JSON.stringify(enteredValues),
      headers: {
        "Content-Type": "application/json",
      },
    })
```

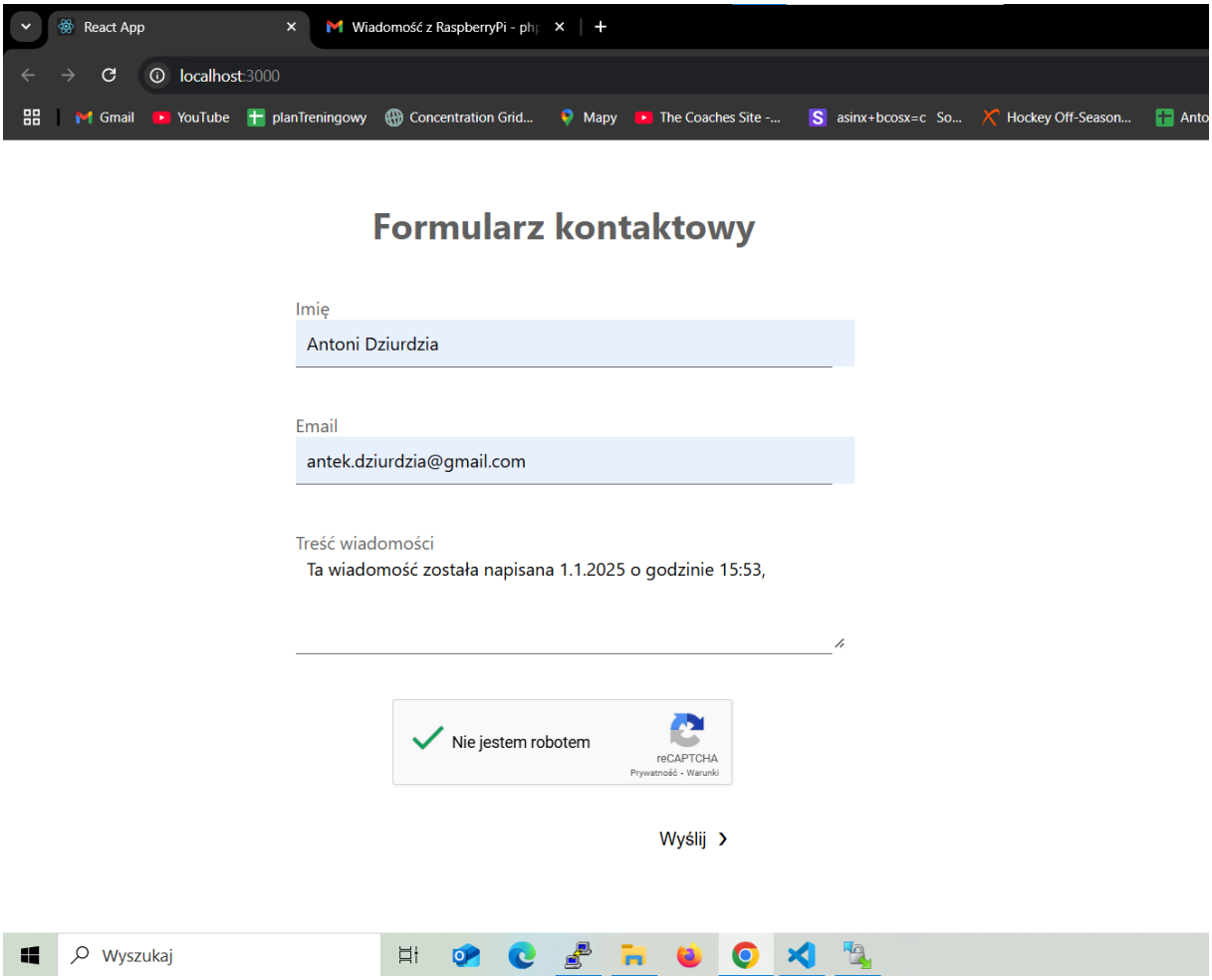
Funkcja wysyłająca dane z formularza na serwer.

Aplikacje uruchomione na serwerze czyli RaspberryPi, są napisane w języku php. Ich zadaniem jest nasłuchiwanie na porcie 80 (domyślny port prtokołu http), a następnie zdekodowanie informacji przychodzącej protokołem. Następnie z wykorzystaniem biblioteki PHPMailer i protokołu SMTP zostaje wysłany email na zdefiniowane konto pocztowe. Na serwerze należy jeszcze umieścić skrypt weryfikujący reCaptche.

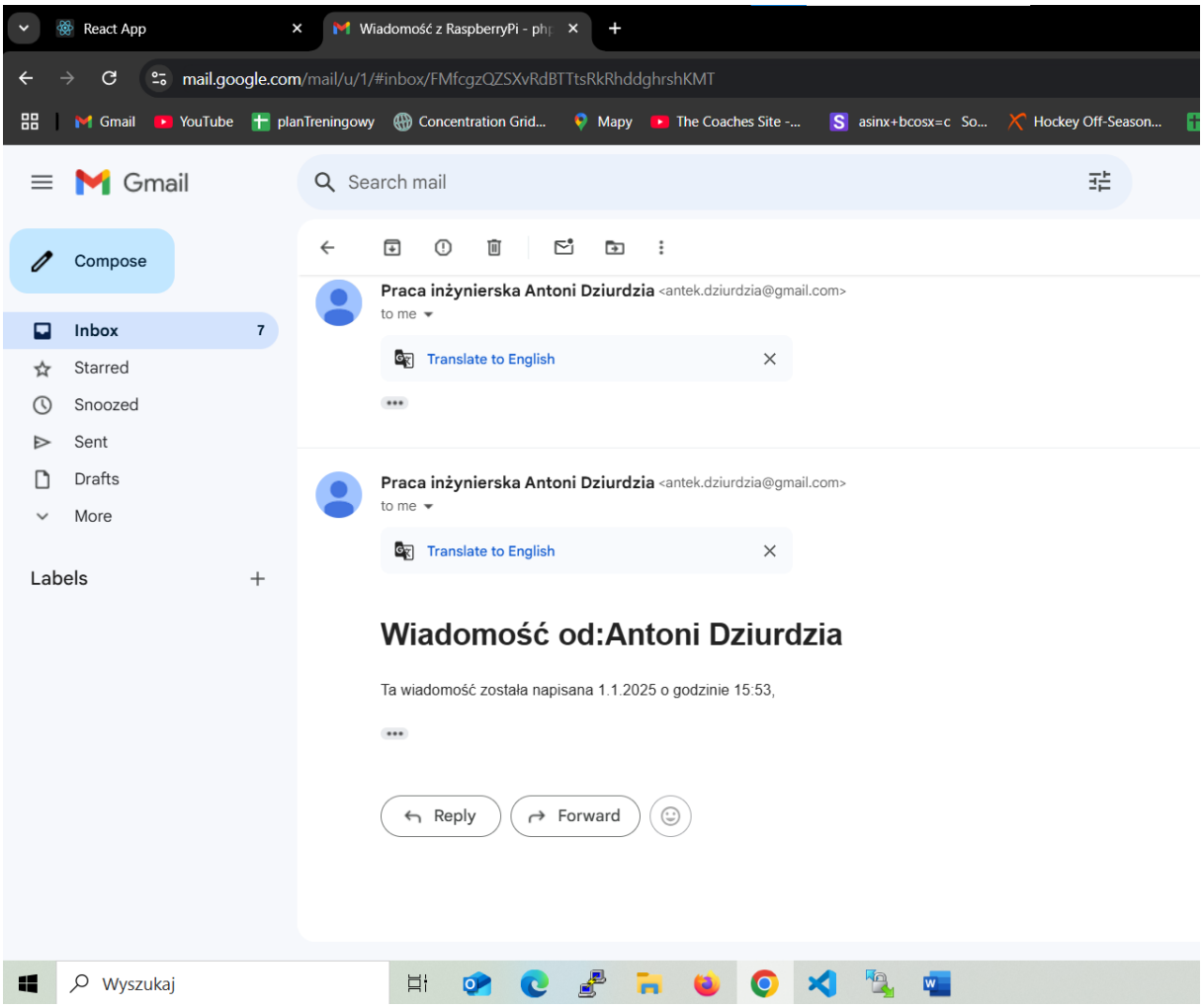
```php
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Headers: *");
header("Content-Type:application/json");

$data = json_decode(file_get_contents('php://input'), true);
$name = $data['name'];
$email = $data['email'];
$text = $data['text'];

use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;
use PHPMailer\PHPMailer\SMTP;

require 'PHPMailer/src/Exception.php';
require 'PHPMailer/src/PHPMailer.php';
require 'PHPMailer/src/SMTP.php';
        try {
            $mail = new PHPMailer();
            $mail->isSMTP();
            $mail->SMTPDebug = SMTP::DEBUG_SERVER;

            $mail->Host = 'smtp.gmail.com';
            $mail->Port = 465;
            $mail->SMTPSecure = PHPMailer::ENCRYPTION_SMTPS;
            $mail->SMTPAuth = true;

            $mail->Username = 'antek.dziurdzia@gmail.com';
            $mail->Password = '***********';
            $mail->CharSet = 'UTF-8';
            $mail->setFrom('no-reply@domena.pl', 'Praca inżynierska Antoni
Dziurdzia');
            $mail->addAddress('phpodc6@gmail.com');
            $mail->addReplyTo('no-reply@domena.pl', '');

            $mail->isHTML(true);
            $mail->Subject = 'Wiadomość z RaspberryPi';
            $mail->Body = '<html>
            <head>
              <title>Nowa wiadomość</title>
            </head>
            <body>
              <h1>Wiadomość od:'. $name .'</h1>
              <p>'. $text .'
              </p>
        <p>Podany adres email: '. $email .' </p> <hr> </body></html>';
            $mail->send();
        } catch(Exception $e) {
            echo "Błąd wysyłania maila: {$mail->ErrorInfo}"; } ?>
```

- *Podsumowanie wyników działania*

Efektem projektu jest serwer www działający na RaspberryPi. Taki serwer może być wykorzystywany do przechowywania danych zbieranych przez RaspberryPi lub tak jak w moim przypadku, odwrotnie, serwer może być usługą dla klienta i za pomocą żądań http obsłużyć aplikacje kliencką. W projekcie wykorzystano dwa języki programowania, php służący do pisania skryptów serwerowych oraz javascript służący do pisania aplikacji przeglądarkowych.



Rysunek 4.8 Frontend aplikacji czyli strona internetowa.



Rysunek 4.9 Wynik działania aplikacji.

4.2. Protokoły szeregowe

4.2.1. CAN

- *Cel i zastosowanie protokołu*

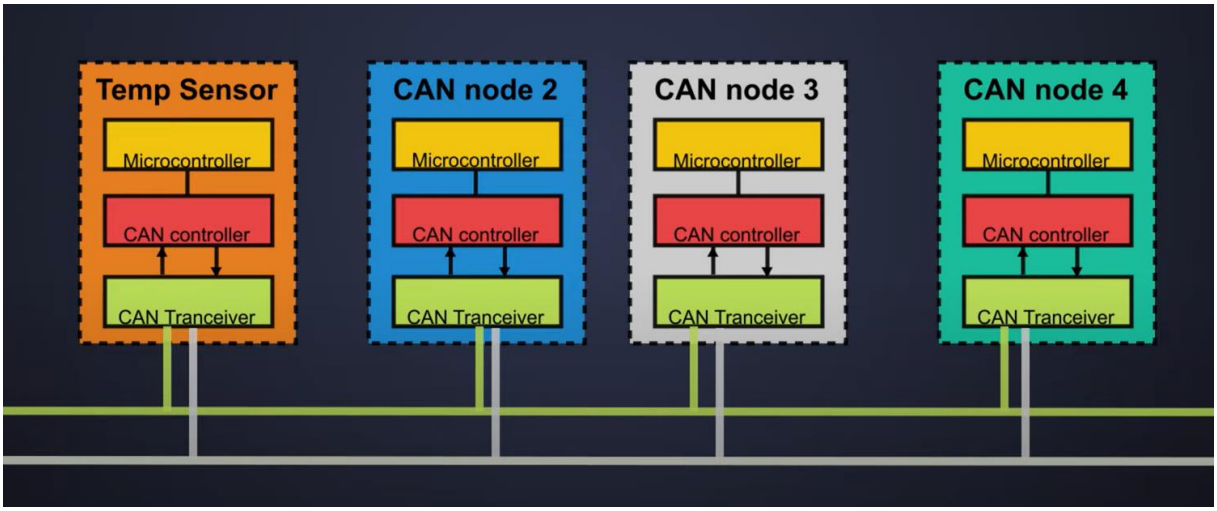
Protokół CAN (Controller Area Network) to standard komunikacyjny opracowany przez firmę Bosch na początku lat 80 XX wieku. Znajduje zastosowanie w przemyśle samochodowym.

- *Opis działania protokołu*

Urządzenia komunikują się na zasadzie broadcast'u. Tylko urządzenie które jest zainteresowane widomością odbiera ją, reszta ignoruje wiadomość. Nie ma jednego mastera, który wysyła wiadomości tak jak to było w modbusie, tutaj każde urządzenie może nadawać. Spowodowana jest tym szansa kolizji nadawania, np. gdy dwa urządzenia zaczynają nadawać wiadomość w tym samym czasie. Ustawieniem wiadomości w kolejce oraz zapobieganiem kolizji i zakłóceń zajmują się specjalne reguły. Protokół zawiera zaawansowane mechanizmy wykrywania błędów. Niezawodność tego protokołu to warunek konieczny, gdyż został on zaprojektowany z myślą o przemyśle motoryzacyjnym, gdzie błędy w komunikacji mogłyby skutkować w ofiarach śmiertelnych. Urządzenia połączone są w Topologii magistrali (bus topology). Dzięki takiemu

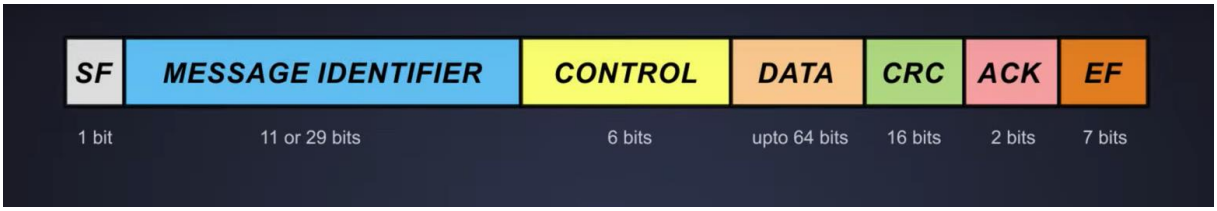
rozwiązaniu łatwiejsze staje się połączenie kablami wielu urządzeń w samochodzie. Magistrala CAN nie posiada struktury hierarchicznej. Mamy w niej rodzaj transmisji szeregowej, asynchronicznej. Zaletą takiej sieci jest fakt, że jeżeli jedno urządzenie zostanie odłączone, wskutek np. awarii to reszta urządzeń powinna wciąż dobrze funkcjonować. Jeżeli w samochodach wykorzystano by np. topologie gwiazdy, w której wszystkie urządzenia podłączone są do jednego kontrolera, to w przypadku awarii kontrolera, unieruchomiona zostałaby cała sieć. Jest to pierwszy protokół w tej pracy, który nie ma hosta, lub mastera. CAN jest systemem half duplex co oznacza, że tylko jedno urządzenie może nadawać do magistrali w określnym momencie. Magistrala korzysta z dwóch linii. Sygnał jest wysyłany w sposób różnicowy, tzn. stan niski oraz wysoki są interpretowane na podstawie różnicy pomiędzy dwoma sygnałami, daje ta dużą odporność na zakłócenia gdyż obie linie poddawane tym samym zakłóceniom nie zmieniają różnicy pomiędzy sobą.

Każde urządzenie podpięte do magistrali CAN, składa się z mikrokontrolera, kontrolera i Transceivera, kontroler dodaje do ramki dodatkowe dane takie ID wiadomości oraz funkcja CRC służąca do wykrywania błędów w transmisji danych.



Rysunek 4.10 Schemat magistrali CAN

CAN Controller wysyła dane do CAN Transceiver wtedy gdy wolna jest magistrala, tzn. kiedy Transceiver nie otrzymuje w danym momencie żadnych danych.



Rysunek 4.11 Struktura ramki danych CAN

- SF to start ramki.
- Message ID, definiuje priorytet ramki. W przypadku gdy dwa urządzenia zaczną nadawać jednocześnie, wygra to z wyższym priorytetem (wyższy priorytet oznacza niższy nr ID).
- Controller zawiera informacje ile będzie informacji w ramce, służy do wykrywania błędów.
- Informacje przekazywane do innych urządzeń przechowywane są w polu Data.
- CRC to funkcja służąca do wykrywania błędów powstałych podczas transmisji.
- ACK to pole, w którym Odbiorca potwierdza Nadawcy, że otrzymał dane.
- EF sygnalizuje koniec ramki.

Przypomnijmy jeszcze raz, że w protokole CAN używany jest sygnał różnicowy, tzn bity 0 i 1 są reprezentowane za pomocą różnicy napięcia pomiędzy dwoma liniami CAN_High i CAN_Low.

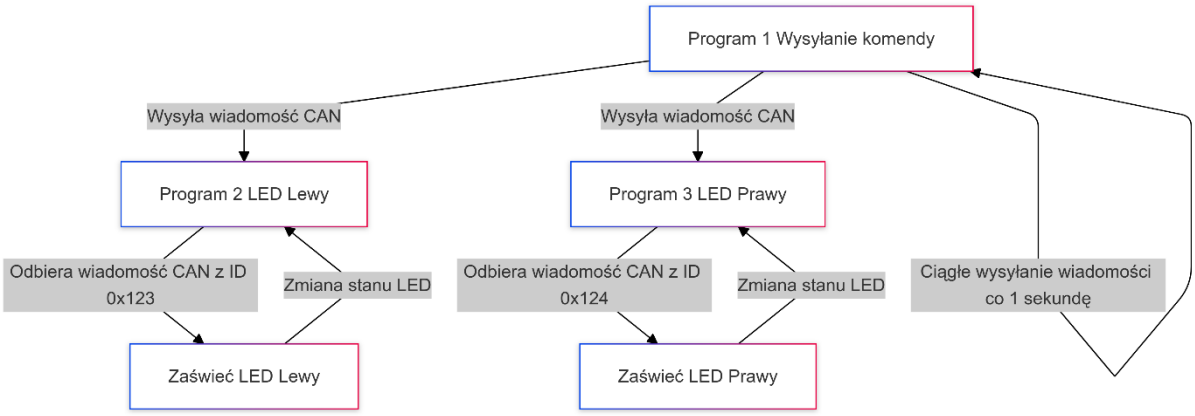
Rysunek 4.12 Po lewej stronie sygnał pojedynczej linii, po prawej sygnał różnicowy.

Warto zwrócić uwagę, że w protokole CAN bit 0 reprezentowany jest przez bit dominujący (duża różnica napięć) natomiast bit 1 reprezentowany jest przez bit recesywny (mała różnica napięć).

System arbitracji polega na tym, że urządzenia podłączone do magistrali CAN same decydują, które ma pierwszeństwo w dostępie do sieci w danym momencie. Służy do tego pole Message Identifier. Jak już wspomniałem im mniejszy nr ID tym wyższy priorytet, dzieje się tak dlatego, że niższy numer w systemie binarnym zawsze będzie miał zero pierwszy (na bardziej znaczącym bicie), 0 jest w protokole CAN bitem dominującym, więc wygrywa z bitem recesywnym. Każdy z węzłów, który próbuje nadawać wiadomość jednocześnie nasłuchuje magistrali. Jeżeli na magistrali pojawi się numer o innym ID (niższym ID) niż nr ID aktualnie nadawanej przez niego wiadomości, to węzeł zaprzestaje nadawanie.

- *Opis stworzonego projektu*

W ramach projektu stworzono wirtualną magistrale CAN-BUS. Umożliwia to biblioteka pyhon-can oraz wirtualny interfejs CAN vcan0. Stworzyłem na raspberryPi trzy programy, symulujące 3 urządzenia podpięte do magistrali, a w rzeczywistości reprezentujące 3 różne procesy. Pierwszy program wysyła paczkę z danymi, dwa pozostałe nasłuchują. Nasłuchujące programy po otrzymaniu przeznaczonej dla nich paczki uruchamiają migającą diodę. System ma przypominać lewy i prawy kierunkowskaz samochodu.



Rysunek 4.13 Flowchart działania aplikacji

- *Kod źródłowy, fragmenty z objaśnieniami*

send_blink_command.py (program wysyłający informacje)

```
import can
import time

def send_blink_command():
    bus = can.interface.Bus(channel='vcan0', bustype='socketcan')
    msg = can.Message(arbitration_id=0x123, data=[0x01], is_extended_id=False)
    try:
        while True:
            bus.send(msg)
            time.sleep(1)
    except KeyboardInterrupt:
        pass

if __name__ == "__main__":
    send_blink_command()
```

W obiekcie klasy can za pomocą metody interface ustawiamy wirtualną linię vcan0. Bustype socketcan odnosi się do zestawów rozszerzeń gniazd sieciowych w systemie Linux, które umożliwiają komunikację przez magistralę CAN za pomocą gniazd sieciowych, podobnie jak w przypadku TCP/IP.

Metoda Message wysyła wiadomość z ID 0x123, oraz z danymi, uzupełniamy też pole is_extended_id mówiące o tym, że wystarczy nam 11 bitowe pole ID, a nie rozszerzone 22 bitowe (w naszym systemie 3 procesów 11 bitów w zupełności wystarczy).

Blink_left_led.py (program obsługujący lewy kierunkowskaz)

```
import RPi.GPIO as GPIO
import can
import time

LED_LEFT = 21

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_LEFT, GPIO.OUT)

def blink_left_led():
    bus = can.interface.Bus(channel='vcan0', bustype='socketcan')
    try:
        while True:
            message = bus.recv()
            if message.arbitration_id == 0x123 and message.data[0] == 0x01:
                GPIO.output(LED_LEFT, GPIO.HIGH)
                time.sleep(0.5)
                GPIO.output(LED_LEFT, GPIO.LOW)
                time.sleep(0.5)
    except KeyboardInterrupt:
        pass
    finally:
        GPIO.cleanup()

if __name__ == "__main__":
    blink_left_led()
```

W tym programie widzimy, że jeżeli ID wiadomości będzie równe 0x123 oraz dane będą równe 0x01 to należy uruchomić diodę.

blink_right_led.py (program obsługujący prawy kierunkowskaz)

```python
import RPi.GPIO as GPIO
import can
import time

LED_RIGHT = 27

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_RIGHT, GPIO.OUT)

def control_right_led():
    bus = can.interface.Bus(channel='vcan0', bustype='socketcan')
    try:
        while True:
            message = bus.recv()
            if message.arbitration_id == 0x124 and message.data[0] == 0x01:
                GPIO.output(LED_RIGHT, GPIO.HIGH)
                time.sleep(0.5)
                GPIO.output(LED_RIGHT, GPIO.LOW)
                time.sleep(0.5)
            else:
                GPIO.output(LED_RIGHT, GPIO.LOW)
    except KeyboardInterrupt:
        pass
    finally:
        GPIO.cleanup()

if __name__ == "__main__":
    control_right_led()
```

Program blink_right_led.py nasłuchuje na wiadomość o ID 0x124. Jeżeli taka pojawi się na linii to program uruchomi prawą diodę.

- *Podsumowanie wyników działania*

Warto zauważyć, że w całym systemie lewa dioda otrzymała wyższy priorytet od diody prawej, to oznacza że w wypadku gdy dwa procesy zaczęłyby wysyłać wiadomość o uruchomieniu każdej z diod w tym samym czasie, to na linii wygrałoby ID diody lewej.

Protokół CAN to pierwszy protokół w tej pracy, który do komunikacji nie używa adresów urządzeń docelowych. Definiowane są numery ID wiadomości na które poszczególne urządzenia oczekują.

4.2.2. Bluetooth

- *Cel i zastosowanie protokołu*

Bluetooth to zaprojektowany 1994 przez firme L.M.Ericssona protokół bezprzewodowy. Nazwa pochodzi od przydomku króla Wikingów Haralda Sinozębnego, który połączył (tzn. podbił) Danie

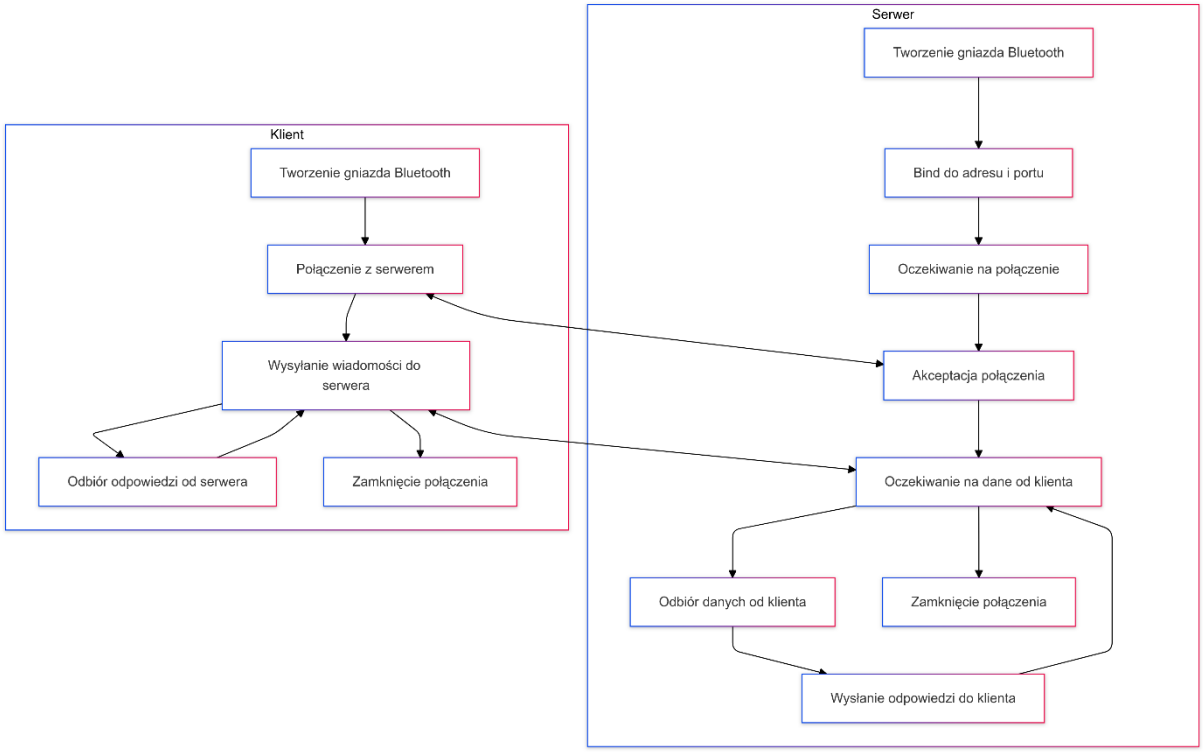
i Norwegię nie używając do tego kabli. Służy do wymiany danych na krótkie odległości, maksymalnie do 100m, za pomocą fal radiowych. Został stworzony w celu umożliwienia komunikacji między urządzeniami takimi jak telefony komórkowe, słuchawki oraz systemy wbudowane.

- *Opis działania protokołu*

Protokół Bluetooth różni się od omawianego wcześniej TCP/IP. Po pierwsze w komunikacji nie odgrywa roli abstrakcyjny adres IP. Zamiast tego urządzenia widzą swoje adresy MAC, które w technologii bluetooth nazywają się BD_ADDR (Bluetooth Device Adress). Adres ten zapewnia unikalną identyfikacje urządzeń. Bluetooth wspiera topologię typu master-slave. Jedno urządzenie (master) może komunikować się z wieloma urządzeniami (slaves).

- *Opis stworzonego projektu*

W celu zbadania technologii stworzono prosty projekt umożliwiający czat między dwoma urządzeniami. W tym celu stworzono dwa programy klient i serwer. Program klient uruchomiono na RaspberyPi natomiast serwer na Laptopie z systemem Windows.



Rysunek 4.14 Flowchart działania aplikacji

- *Kod źródłowy, fragmenty z objaśnieniami*

Program serwer:

```python
import socket


server = socket.socket(socket.AF_BLUETOOTH, socket.SOCK_STREAM,
socket.BTPROTO_RFCOMM)
server.bind(("9c:da:3e:e5:3a:75", 4))
server.listen(1)


client, addr = server.accept()


try:
    while True:
        data = client.recv(1024)
        if not data:
            break
        print(f"Message: {data.decode('utf-8')}")
        message = input("Enter message:")
        client.send(message.encode("utf-8"))
except OSError as e:
    pass


client.close()
server.close()
```

Program klient:

```
import socket

client = socket.socket(socket.AF_BLUETOOTH, socket.SOCK_STREAM,
socket.BTPROTO_RFCOMM)

client.connect(("9c:da:3e:e5:3a:75", 4))


try:

    while True:

        message = input("Enter message: ")

        client.send(message.encode("utf-8"))

        data = client.recv(1024)

        if not data:

            break

        print(f"Message: {data.decode('utf-8')}")

except OSError as e:

    pass


client.close()
```

Przyjrzyjmy się linii, w której tworzymy gniazdo.

```
import socket

client = socket.socket(socket.AF_BLUETOOTH, socket.SOCK_STREAM,
socket.BTPROTO_RFCOMM)
```

‘AF’ oznacza „Adress Family”. ‘AF_BLUETOOTH’ wskazuje, że gniazdo będzie używane do komunikacji Bluetooth.

‘SOCK_STREAM’ określa typ gniazda. W tym przypadku wskazuje, że gniazdo będzie strumieniowe, co oznacza połączenie oparte na strumieniach bajtów, podobne do połączenia TCP w sieciach IP. W kontekście Bluetooth, ten typ gniazda jest używany dla połączeń RFCOMM.

‘BTPROTO_RFCOMM’ określa protokół bluetooth, który będzie używany. Tutaj używany jest RFCOMM, który emuluje port szeregowy RS-232. RS-232 to standard komunikacji szeregowej, w którym urządzenia są połączone fizycznym kablem, a dane są wysyłane szeregowo, bit po bicie, przez port szeregowy (COM port). RFCOMM naśladuje sposób w jaki dane są przesyłane przez port szeregowy RS-232. To oznacza, że aplikacje mogą korzystać z tych samych instrukcji do komunikacji przez RFCOMM, jakie używałyby do komunikacji przez RS-232. Znakomite wykorzystanie istniejących już wcześniej zasobów.

- *Podsumowanie wyników działania*

W takim połączeniu serwer pełni rolę slave. Czeka na połączenie przychodzące (‘server.accept()’), co oznacza, że nie inicjuje połączeń, ale odbiera je od innych urządzeń. Klient pełni rolę master. Inicjuje połączenie do serwera (‘client.connect()’), co oznacza, że to on rozpoczyna komunikację.

Zarówno w programie serwera i klienta widzimy wykorzystanie adresu MAC, serwer binduje się do adresu, natomiast klient próbuje się z nim połączyć, drugim argumentem funkcji bind oraz connect jest channel, odpowiednik portu z protokołu TCP/IP.

Jak widać generalna struktura programu jest podobna do programów serwer klient korzystających z protokołów TCP/IP. Tworzymy gniazdo, serwer binduję się do pewnego adresu natomiast klient próbuje się z tym adresem połączyć. Wykorzystywanie kanałów tak jak i portów, pozwala na jednoczesną komunikację wielu procesów. Po nawiązaniu połączenia otwiera się kanał przesyłu danych między urządzeniami i za pomocą send(), recv() można obsłużyć prostą komunikację. Na koniec należy zamknąć gniazdo.

4.3. Protokoły wysokiego poziomu

4.3.1. OPC UA

- *Cel i zastosowanie protokołu*

Protokół OPC UA (Open Platform Communications Unified Architecture) został opracowany przez OPC Fundation. Protokół jest wynikiem wspólnej pracy liderów rynku przemysłowego, ma na celu ujednolicenie komunikacji pomiędzy sterownikami urządzeniami i systemami zarządzającymi w fabrykach. Wcześniejsze wersje OPC działały tylko na Windowsie oraz nie posiadały interfejsu internetowego. Te mankamenty OPC Classic przyczyniły się do wypuszczenia specyfikacji OPC UA w 2008 roku.

- *Opis działania protokołu*

Wymieńmy kilka podstawowych cech OPC UA:

- Działa na wszystkich popularnych systemach operacyjnych, więc pasuje do aplikacji cross-platform.
- Wspiera komunikacje przez internet.
- Dane wysyłane są w postaci tak zwanych „nodes”, które można porównać do obiektów w standardowym programowaniu. Pozwala to na dokładniejszą i obszerniejszą informacje na temat urządzeń oferujących dane.

- Ułatwienie integracji. Dzięki OPC UA, obejść można kaskadową strukturę przekazywania danych w przemyśle. Urządzenia takie jak czujniki nie muszą kontaktować się ze sterownikami które następnie wysyłają dane to urządzeń jak SCADA czy HMI. OPC UA ma na celu ujednolicenie komunikacji to znaczy, sprawienie żeby każdy mógł porozumieć się z każdym, najwyższe i najważniejsze systemy w sieci mogą komunikować się bezpośrednio z najniższymi w hierarchii komponentami zdolnymi do stworzenia Serwera OPC UA.

OPC UA posiada dwa różne sposoby wymiany danych:

- Binarnie zakodowane wiadomości wysłane przez protokół TCP. Ten sposób zapewnia szybkość i powinien zostać wykorzystany tam gdzie priorytetem jest performance.
- Wiadomości kodowane w języku XML wysyłane protokołem HTTPS. To przyjazny dla użytkownika sposób wymiany informacji.

Model komunikacji to Serwer-Klient. Dla przykładu: pompa to serwer który udostępnia swoje informacje klientowi który jest uruchomiony na PC i zbiera danych w celu wizualizacji stanu fizycznego maszyny.

Protokół jest szansą na integracje systemów przemysłowych. Próby ujednolicenia komunikacji w przemyśle to duży krok na przód. Może to jednak nie być na rękę producentom robotów, sterowników, maszyn, którzy specjalnie implementują własne protokoły na swoich urządzeniach w celu zmuszenia przedsiębiorcy do dalszych zakupów w obrębie ich oferty produktowej.

W OPC UA „nodes” są podstawowymi elementami struktury informacyjnej. Każdy inny element informacyjny jest zbudowany na bazie „node”. Każdy pojedynczy node zawiera informacje o jego powiązaniu z innymi „nodes”.

- *Opis stworzonego projektu*

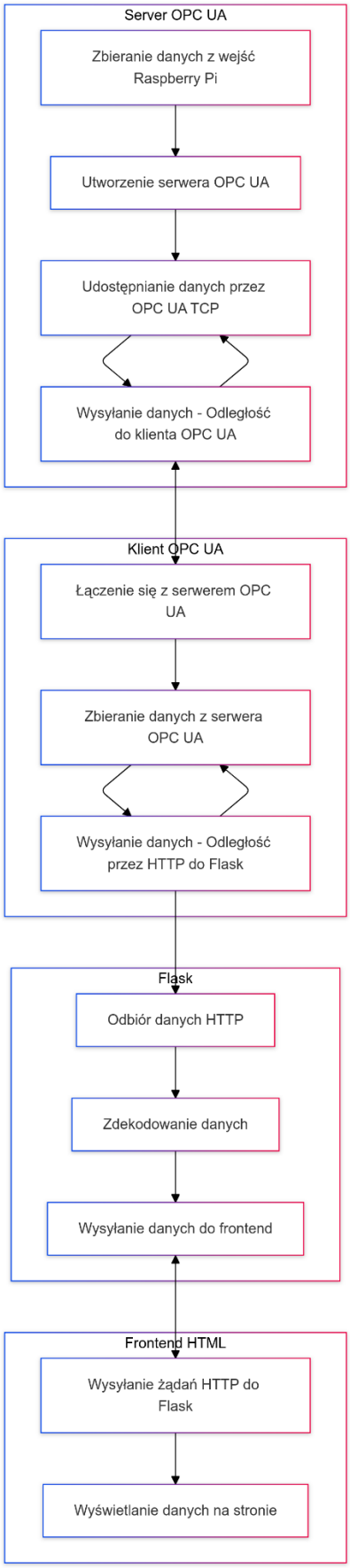
Projekt to 4 programy napisane w języku python mające na celu wymianę informacji pomiędzy Raspberry Pi a stroną internetową. Do komunikacji używamy protokołu OPCUA oraz HTTP.

Pierwszy program zbierający dane z wejść RaspberryPi oraz tworzący serwer OPC UA, który udostępnia dane.

Program używa binarnego kodowania wiadomości, korzysta z protokołu TCP. OPC UA umożliwia również wysyłanie wiadomości protokołem https. Niestety bibliotka python-opcua nie obsługuje tego typu przesyłania danych dlatego zdecydowałem się na wykorzystanie protokołu TCP.

Drugi program tworzy klienta OPCUA, który zbiera dane z serwera utworzonego na RaspberryPi a następnie wysyła je protokołem HTTP.

Kolejne dwa programy to kolejno: aplikacja w frameworku pythona flask. Służy ona do odbioru i zdekodowania żądania http, oraz strona html, tak zwany frontend, który komunikuje się z flaskiem i wyświetla na stronie internetowej zdekodowane informacje wysłane z raspberryPi.



Rysunek 4.15 Flowchart działania aplikacji

- *Kod źródłowy, fragmenty z objaśnieniami*

```
from opcua import Server
import time
import RPi.GPIO as GPIO

def get_distance():
      #celowo nie umieszczam tu kodu odpowiedzialnego za odbiór danych z wejść
      #GPIO, podobny kod można znaleźć w innych projektach w tej pracy
    return distance

server = Server()
url = "opc.tcp://0.0.0.0:4840"
server.set_endpoint(url)

name = "DistanceOPCUAServer"
addspace = server.register_namespace(name)

node = server.get_objects_node()
param = node.add_object(addspace, "Parameters")

distance = param.add_variable(addspace, "Distance", 0.0)
distance.set_writable()

server.start()
print("Serwer OPC UA uruchomiony na {}".format(url))

try:
    while True:
        dist = get_distance()
        print(f"Odległość: {dist} cm")
        distance.set_value(dist)
        time.sleep(1)
except KeyboardInterrupt:
    server.stop()
    GPIO.cleanup()-9
```

- **serwer = Server()** tworzy nową instancję serwera OPC UA przy użyciu biblioteki **'python-opcua'**
- **url = „opc.tcp://0.0.0.0:4840"** oraz **server.set_endpoint(url)** endpoint definiuje jak klienci mogą połączyć się z serwerem. Za pomocą zmiennej url ustawiamy endopint z protokołem **'opc.tcp'** nasłuchujący na wszystkich interfejsach sieciowych (**'0.0.0.0'**) na porcie **'4840'**.

- **addspace = server.register_namespace(name)** przypisuje do przestrzeni nazw zmienną „name”, przestrzeń nazw (z ang. namespace) służy do identyfikacji węzła.
- **node = server.get_objects_node()** tworzy główny węzeł, który jest punktem wyjścia do dodawania kolejnych węzłów.
- **param = node.add_object(addspace, "Parameters")** dodaje nowy obiekt do głównego węzła, ten obiekt przechowywany jest w zmiennej param.
- **distance = param.add_variable(addspace, "Distance", 0.0)** dodaje zmienną „Distance” do obiektu „Parameters” oraz ustawia jej wartość początkową.
- **distance.set_writable()** umożliwia modyfikacje zmiennej klientom OPC UA.
- **server.start()** uruchamia server.

```
import requests
import time
from opcua import Client

url = "opc.tcp://localhost:4840"
client = Client(url)
client.connect()

while True:
    dist = client.get_node("ns=2;i=2").get_value()
    response = requests.post("http://localhost:5000/update_distance",
json={"distance": dist})
    print("wysłano")
    time.sleep(1)

client.disconnect()
```

- **client.get_node(**„ns=2;i=2”**).get_value()** **„ns”** oraz **„i”** to parametry służące do identyfikacji „node”, który przechowuje interesujące nas dane. Ns to skrót od namespace a **„i”** to identyfikator wskazujący na liczbę całkowitą.

- *Podsumowanie wyników działania*

Projekt wiąże ze sobą dwa protokoły, OPCUA (działający na TCP), oraz http. Docelowo samo OPCUA powinno dawać możliwość korzystania z protokołu http, wtedy system wymiany danych między malinką a przeglądarką internetową powinien polegać na samym OPCUA. Niestety ze względu na ograniczenia biblioteki, nie mogłem zbadać takiego rozwiązania.

4.3.2. API

- *Cel i zastosowanie protokołu*

API (Application Programming Interface) nie jest samo w sobie protokołem. API to zbiór protokołów i procedur. Głównym celem API jest ułatwienie programistom korzystania z istniejących już systemów. Dla przykładu Google Maps może udostępniać API swoich map tak żeby inne aplikacje mogły z nich korzystać

- *Opis działania protokołu*

API udostępnia procedury, którymi możemy korzystać z zasobów różnych serwisów, natomiast komunikacja za pomocą protokołów jest wbudowana w API i działa automatycznie.

- *Opis stworzonego projektu*

Projekt oparto o API organizacji openAI. Raspberry Pi zostało przekształcone w inteligentne urządzenie potrafiące odpowiadać na pytania, działające jak popularny chatGPT. Niezbędnymi krokami są: założenie konta na platformie openAI, uzyskanie sekretnego klucza API, doładowanie konta środkami. Programistyczna strona projektu polegała na pobraniu modułu openAI na systemie raspberryPi. Żeby go pobrać należało najpierw utworzyć wirtualne środowisko. Wirtualne środowiska w Pythonie to izolowane środowiska pozwalające na zarządzanie bibliotekami i modułami niezależnie od systemu globalnego. Ich zaletą jest izolacja, pozwalająca uniknąć konfliktów między różnymi projektami. Środowisko takie tworzy się za pomocą polecenia:

```
python3 -m venv myenv
```

Po utworzeniu środowiska aktywujemy je:

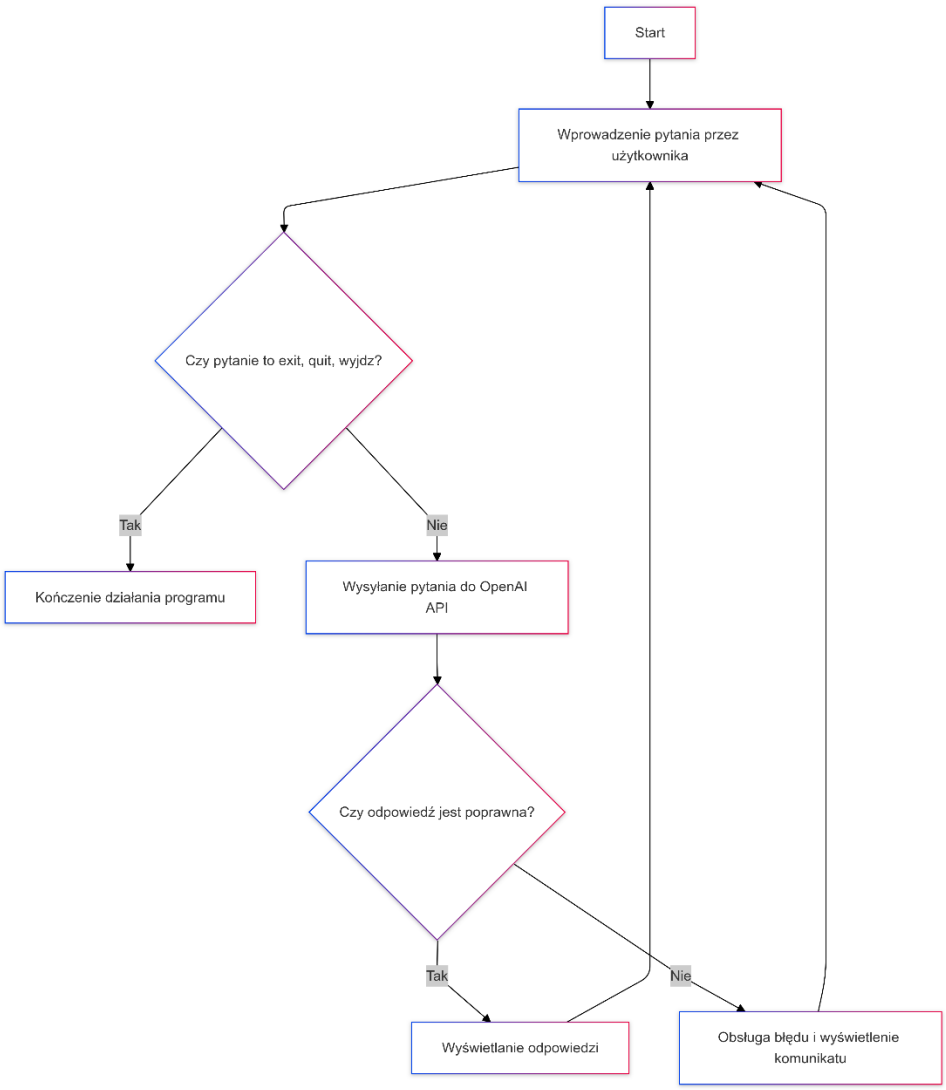
```
source myenv/bin/activate
```

A na koniec deaktywujemy:

```
deactivate
```

W wirtualnych środowiskach instalujemy pakiety np. przykład za pomocą narzędzia pip.

```
pip install openai
```



Rysunek 4.16 Flowchart działania programu

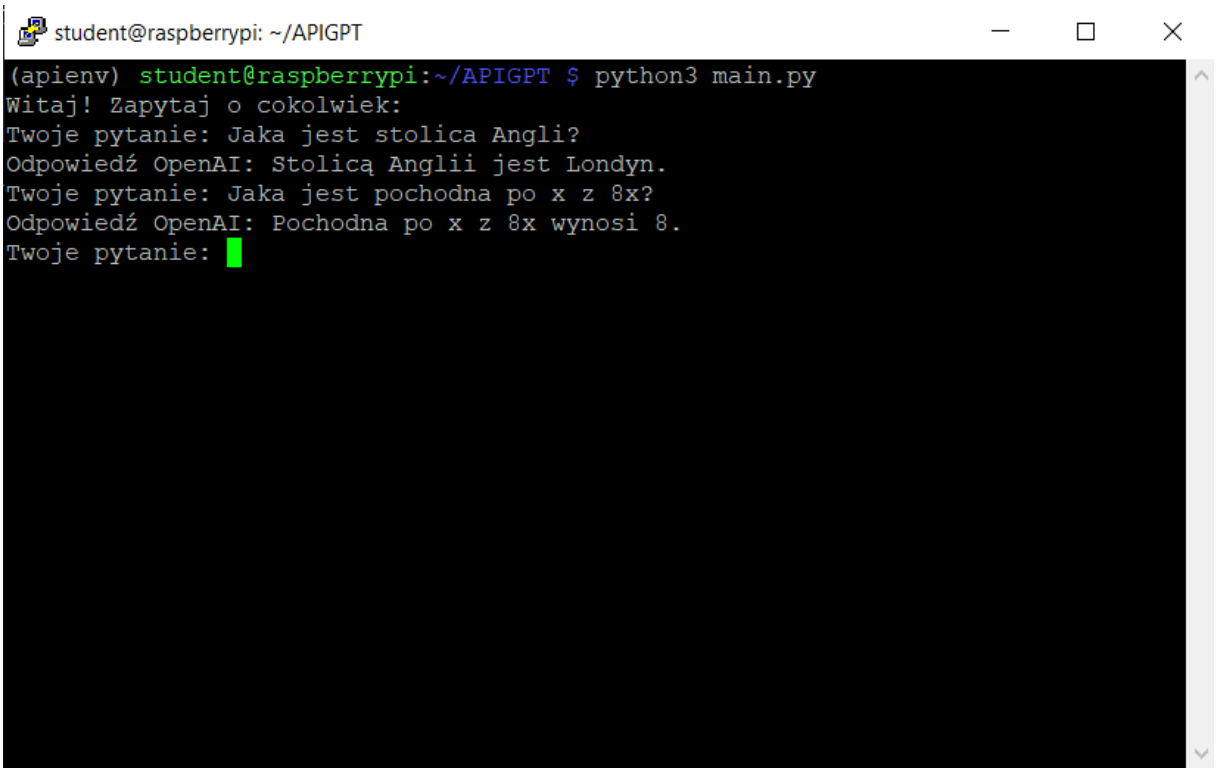
- *Kod źródłowy*

Kod programu main.py korzystającego z API openAI:

```
import openai

openai.api_key = "sk-proj-dBHPpsDzHQMzHitbxn4RbUyJkZ5WJGzTNpaYEY-
aTDgHvjq25KXU7TIdlzUa8W6BfY9h97vKbL2T3BlbkFJAecOX8yfhz1nBkj7IhJcOODmil9s-_SKo-
-rk7PAhlB8D91x8s5tP9EJc5uR9GtVdrtZGm2nAA"

def ask_openai(question):
    try:
        response = openai.ChatCompletion.create(
            model="gpt-4",
            messages=[
                {"role": "user", "content": question}
            ],
            temperature=0.7,
            max_tokens=150
        )

        return response.choices[0].message['content'].strip()
    except Exception as e:
        return f"Błąd podczas korzystania z OpenAI API: {e}"

if __name__ == "__main__":
    print("Witaj! Zapytaj o cokolwiek:")
    while True:
        question = input("Twoje pytanie: ")

        if question.lower() in ["exit", "quit", "wyjdz"]:
            print("Kończę działanie programu.")
            break

        answer = ask_openai(question)
        print(f"Odpowiedź OpenAI: {answer}")
```

- *Podsumowanie wyników działania*

Wynik działania programu:



Konsola połączona protokołem SSH z RaspberryPi, pokazująca efekt działania programu

API samo w sobie to nie protokół. API to technologia wysokiego poziomu, wykorzystywana przez developerów w celach biznesowych. Udostępniając systemy lub dane w API, organizacje zazwyczaj pobierają opłaty związane z ich użytkowaniem. Tworzenie API we własnych projektach może nie tylko ułatwić życie innym programistom ale także przynieść nam zysk zarobkowy.

5. Protokoły komunikacyjne – opracowania teoretyczne

5.1. Protokoły przemysłowe

5.1.1. Profibus

- *Cel i zastosowanie protokołu*

Profibus (PROcess Field BUS) to standard sieci przemysłowej czasu rzeczywistego. Protokół został opracowany a latach 80 lecz wciąż znajduję szerokie zastosowanie w przemyśle. Jest wykorzystywany do sterowania procesami technologicznymi i liniami produkcyjnymi, najczęściej łączy sterownik PLC z maszyną.

- *Opis działania protokołu*

Protokół Profibus wykorzystuje model Master – Slave. W ciągły sposób master odpytuje i oczekuje odpowiedzi aby sprawdzić stan wszystkich urządzeń typu slave. Sieć profibus ma magistrale liniową, jest możliwe zakupienie specjalnych rozwiązań, tak zwanych reapeat'erów, które umożliwiają stworzenie sieci w topologii gwiazdy. Sieć oparta jest najczęściej na interfejsie RS-485. Na magistrali dostępny jest sygnał różnicowy, który dzięki swojej odporności na zakłócenia znakomicie sprawdza się w warunkach przemysłowych. Na końcu i na początku

magistrali znajdują się terminatory, czyli rezystory zapobiegające odbiciom sygnału. W sieci możemy zaadresować 128 urządzeń, w czym 124 adresy mogę być wykorzystane na slave'y. W rzeczywistości gdy stawiamy sieć profibus na interfejsie RS-485 to mamy do czynienia z ograniczeniami w segmentach. Wynika to z właściwości interfejsu RS-485. Każdy segment może zawierać maksymalnie 32 urządzenia. Segmenty można ze sobą łączyć i przedłużać. Prędkość transmisji zależy od długości segmentów.

Można wyróżnić **PROFIBUS-DA** oraz **PROFIBAS-PA**. Różnią się one architekturą sieci. W PROFIBUS-DA (Decentralized Periferials) odłączamy moduły wejść/wyjść od sterownika PLC. Umieszczamy je w bliskiej odległości do czujników z którymi wymieniają one informacje. Moduły ze sterownikiem łączymy kablem RS-485. Warto pomyśleć o zastosowaniu dwóch kabli, jednego o charakterze awaryjnym na wypadek awarii. PROFIBUS-PA (Process Automation) pozwala na pozbycie się modułów wejść/wyjść, oraz kabli podłączający każdy z czujników osobno do modułu. I wykorzystać jedną linie, magistrale do której podpięte będą wszystkie czujniki. Przed sterownikiem PLC należy umieścić specjalny konwerter, który przekształci PROFIBU-PA na PROFIBUS-DP, gdyż większość sterowników PLC nie posiada wejść rozumiejących PROFIBUS-PA.

- *Przykładowe zastosowania w praktyce*

Profibus wykorzystywany jest przez starsze sterowniki SIEMENS Serii s7 300/400. Przykładowe połączenie wymagające komunikacji Profibus to: sterownik – panel HMI – falownik.

5.1.2. Profinet

- *Cel i zastosowanie protokołu*

Profinet (Process Field Net) to protokół stosowany w automatyce przemysłowej. Jest to otwarte rozwiązanie co oznacza, że może być wykorzystywany przez każdego producenta. Daje to szanse na interoperacyjność pomiędzy urządzeniami różnych dostawców.

- *Opis działania protokołu*

Protokół Profinet wykorzystuje standard Ethernet do przesyłania danych między różnymi urządzeniami. Dlatego jest tani w wdrożeniu, szybszy, bardziej wydajny i wspiera bardziej skomplikowanie topologie niż PROFIBUS. W warunkach przemysłowych często wybierana jest specjalnie ekranowana wersja przewodu. W sieci profinet master jest nazywany kontrolerem i jest połączony do wielu węzłów sieci za pomocą podwójnej linii sygnałowej, co daje możliwość full-duplex czyli wymiany danych jednocześnie w obu kierunkach. Za pomocą switchy, urządzeń centralnych w topologii gwiazdy, informacje są inteligentnie wysyłane do swojego miejsca docelowego. Switche zapamiętują adresy MAC urządzeń do nich podłączonych, a następnie kojarzą je z odpowiednimi wyjściami, cały proces jest dynamiczny, nie wymaga zatem ręcznego konfigurowania. W sieci profibus możliwa jest obsługa wielu urządzeń w tym samym czasie, co w porównaniu do PROFIBUS zwiększa obciążenie sieci, ale również szybkość. Sieć PROFINET pozwala na konfiguracje czasu aktualizacji każdego urządzenia w sieci, szybkość ta ustalona powinna być w oparciu o cykl pracy sterownika PLC. Switche potrafią również kontrolować

błędy w pakietach. Jeżeli w sieci będą urządzenia wysyłające pakiety w protokole TCP/IP takie jak na przykład kamery przemysłowe to może dojść do opóźnień. Pakiety TCP/IP są o wiele większe od pakietów profinetowych i powoduje spowolnienie sieci. Profinet jest zaprojektowany do komunikacji czasu rzeczywistego, oferuje klasy usług takie jak RT (Realt Time) oraz IRT (Isochronous Real-Time), które umożliwiają deterministyczną komunikację z niskimi opóźnieniami. Oprócz RT oraz IRT Profinet oferuje również komunikacje z wykorzystaniem TCP/IP lub UDP/IP do zadań niewymagających ścisłej komunikacji w czasie rzeczywistym, oraz z wykorzystaniem TSN (Time Sensitive Networking). Wszystkie te klasy mogą współistnieć w jednej instalacji. Usługa Real Time omija warstwy TCP/UDP oraz IP i zostawia tylko warstwę aplikacji i Ethernet, co zmniejsza pakiety do minimum i przyspiesza transport. IRT jest jeszcze szybszy niż RT.

- *Przykładowe zastosowania w praktyce*

Profinet zastępuje Profibus w nowszych sterownikach SIEMENS serii 1200-1500. Przykładowe połączenie to: sterownik – panel HMI – falownik. Różni się od Profibusa sposobem podłączenia. Wszystko zbiega się w Switchu i wykorzystuje interfejs Ethernet.

5.1.3. Modbus RTU

- *Cel i zastosowanie protokołu*

Podstawą różnicą w wersji RTU jest wykorzystywane medium transmisyjne. W TCP korzystano z transmisji ethernetowej, w RTU używamy transmisji szeregowej. Najczęściej używany interfejs do transmisji szeregowej w protokole Modbus RTU to RS485, można znaleźć aplikacje korzystające z RS232, ale jest to coraz rzadziej spotykana metoda.

- *Opis działania protokołu*

Sieć wykorzystująca Modbus korzysta z modelu Slave Master. Urządzenie Slave nigdy nie inicjuje połączeń, urządzenie master może inicjować połączenia, odczytywać oraz zapisywać dane z połączonych z nim urządzeń typu slave. Wszystkie urządzenia w sieci Modbus muszą mieć ustawioną taką samą prędkość przesyłu danych tzw. Baud rate (bits per second). Standardowym ustawieniem jest 9600. Modbus to popularny protokół, który wiele urządzeń rozumie, dlatego może być dobrym rozwiązaniem w wielu aplikacjach. W Modbus RTU tak samo jak w TCP zdefiniowano dwa typy danych cewki oraz rejestry.

- *Przykładowe zastosowania w praktyce*

Modbus RTU łączy moduły ADAM, które zbierają sygnały z czujników na maszynach i przetwarzają na sygnał binarny oraz wysyłają do bazy danych. Takie rozwiązanie używane jest w systemach akwizycji danych czyli opomiarowania, które na bieżąco monitoruje stan produkcji na wszystkich maszynach produkcyjnych.

5.2. Protokoły szeregowe niskiego poziomu

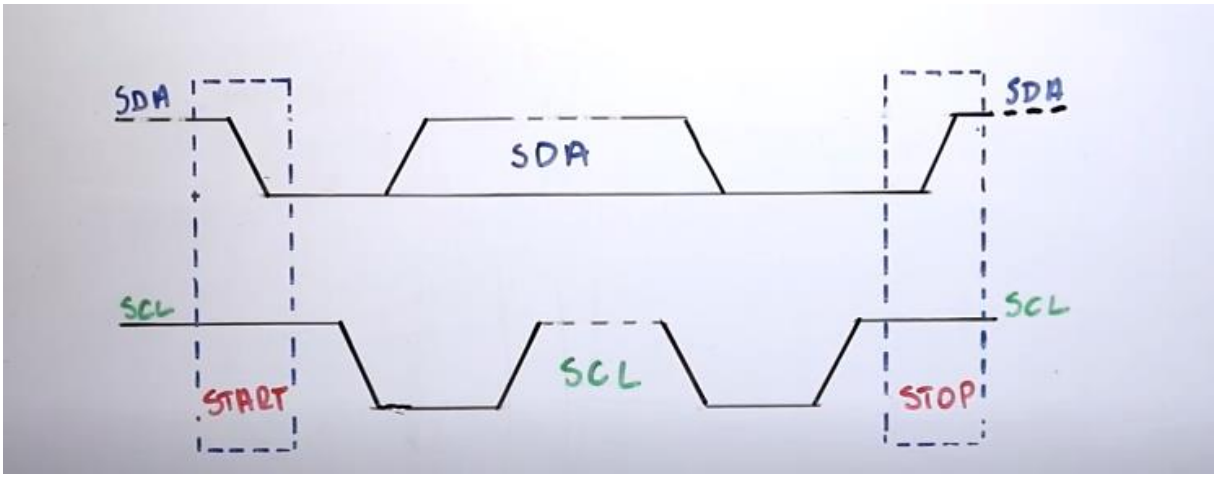
5.2.1. I²C

- *Cel i zastosowanie protokołu*

I²c (inter-integrated circuit) to protokół synchronicznej komunikacji szeregowej. Został stworzony by umożliwić komunikację między układami scalonymi w przemyśle telewizyjnym. Najczęstsze wdrożenia i²c można znaleźć w układach mikroprocesorowych.

- *Opis działania protokołu*

Protokół korzysta z dwuliniowej magistrali, pierwsza linia to SCL (Serial Clock), druga to SDA (Serial Data). Fizycznie jednak potrzeba trzech przewodów bo jeszcze masa jako potencjał odniesienia. I²c przesyła dane szeregowo, synchronicznie dzięki sygnałowi SCL i dwukierunkowo. W protokole mamy komunikacje master-slave, mamy więc jednostkę nadrzędną w postaci mastera. Istnieją przypadki w których na magistrali może być więcej masterów, wtedy specjalne reguły arbitrażu, czyli pierwszeństwa nadawania, pilnują porządku na magistrali. Reguła arbitrażu przypomina tą wykorzystywaną w protokole CAN. Master który pierwszy nada bit zerowy, czyli bit dominujący, przejmuje pierwszeństwo transmisji, drugi po zauważeniu, że przegrał arbitraż, czeka na swoją kolej. Urządzenie nadrzędne (master) jest źródłem sygnału zegarowego SCL. Każde z urządzeń podłączone do magistrali musi mieć swój unikalny adres. Na magistrali mamy logikę dodatnią, to znaczy stan wysoki odpowiada logicznej jedynce. Dane są organizowane jako bajty, czyli grupy 8 bitów. Liczba przesyłanych bajtów nie jest ograniczona, dodatkowo każdy bajt danych musi być potwierdzony przez odbiornik. Gdy sygnał nie jest przesyłany to obie linie magistrali są w stanie wysokim. W protokole i²c istnieje zasada, że gdy na linii SCL stan jest wysoki, to na linii SDA stan musi być stabilny nie może się zmieniać, wyjątkiem są sygnały startu i stopu.



Rysunek 5.1 Sygnały startu i stopu w protokole i²c

Odczyt linii SDA rozpoczyna się w chwili pojawienia się zbocza narastającego na linii SCL. Gdy stan zegara, linii SCL, jest niski, to linia SDA może zmienić swoją wartość w zależności od kolejnego bitu jaki chce przesłać. W każdym takie zegarowym jest przesyłany jeden bit. 9 tak zegarowy służy jako acknowladge bit, czyli potwierdzenie przez odbiornik, odebranie bajtu danych. Podczas tego 9 taktu zegarowego, nadajnik przestaje nadawać i zwalnia magistrale dla odbiornika który może potwierdzić odebranie danych. W starszych wersjach magistrali pierwszy

bajt informacji to 7 bitów adresu urządzenia oraz bit read/write informujący o tym czy dane będą czytane czy nadpisywane, w nowszych wersjach mamy 10 bitowe pole adresu, które pozwala na podłączenie większej liczby urządzeń do sieci. W 7 bitowym adresie urządzenia 4 pierwsze bity to klasa urządzenia, definiujące czy mamy do czynienia z czujnikiem, pamięcią czy mikrokontrolerem, a następne 3 bity to unikalny numer urządzenia. Limitem od strony logicznej jest liczba adresów, a od strony fizycznej to między innymi pojemność magistrali, oraz ograniczony zasięg do kilku metrów.

- *Przykładowe zastosowania w praktyce*

Przykładowym zastosowaniem może być komunikacja systemu wbudowanego z pamięcią EEPROM.

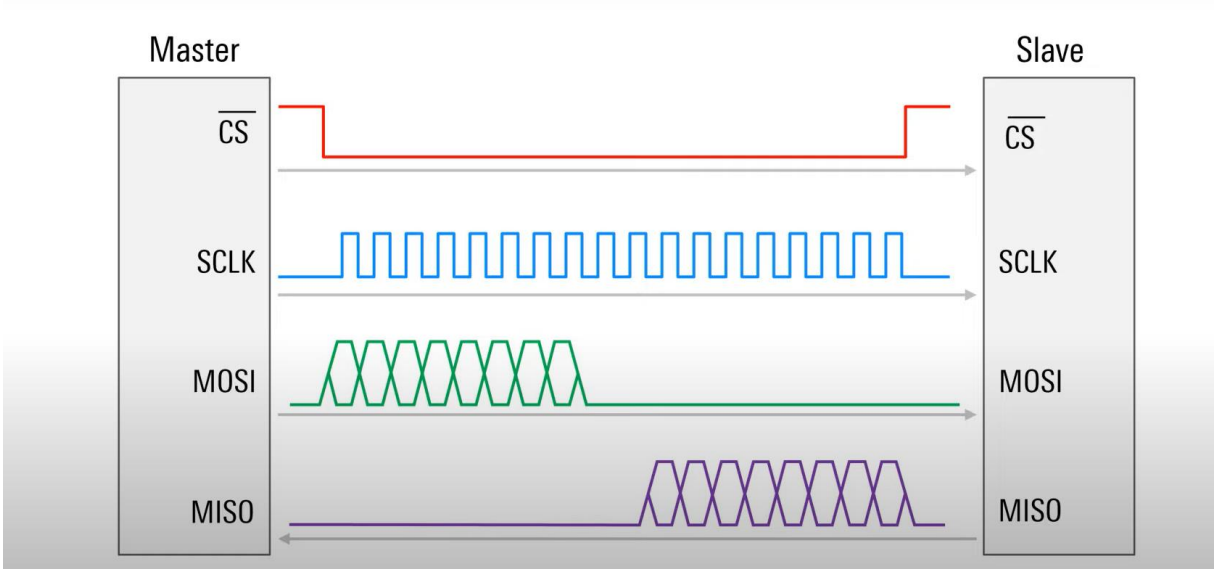
5.2.2. SPI

- *Cel i zastosowanie protokołu*

SPI (Serial Peripherial Interface) to protokół wykorzystywany do komunikacji mikrokontrolerów z urządzeniami peryferyjnymi, takimi jak sensory, pamięci zewnętrzne lub inne mikrokontrolery.

- *Opis działania protokołu*

SPI to szeregowy, synchroniczny prosty i bardzo szybki protokół. Osiąga prędkość transmisji do kilkudziesięciu megabitów na sekundę. To właśnie wysoka wydajność, wyróżnia SPI na tle innych protokołów szeregowych takich jak UART czy i2c. SPI działa na zasadzie master-slave i wspiera wymianę danych w dwóch kierunkach (full-duplex).



Rysunek 5.2 Schemat połączeń interfejsu SPI

W interfejsie SPI używamy 4 linii:

- CS (Chip Select) linia informująca urządzenie Slave, że do niego adresowane są dane. Linia Chip Select jest alternatywą do adresów wykorzystywanych w większości

protokołów. Domyślnie jest w stanie wysokim, a informacja o nadawaniu przez mastera informacji definiowana jest stanem niskim, stąd negacja nad wyjściem/wejściem CS. Koniec transmisji jest sygnalizowany powrotem linii Chip select w stan wysoki. Niestety przy połączeniach większych liczby urządzeń typu slave do mastera, możemy skończyć z dużą liczbą bali odpowiedzialnych za linie CS. Istnieje alternatywna topologia, zwana „daisy chain" lub „cooperative slaves", w której urządzenia typu slave połączone są szeregowo.

- SCLK (Serial Clock) pełni rolę zegara. SPI to protokół synchroniczny konieczne więc do komunikacji jest określenie taktowania przez mastera. Serial Clock może przyjmować w stanie nieaktywnym stan niski lub wysoki (nazywamy to polaryzacją zegara) a bity na liniach MOSI i MISO mogą być odczytywane na zboczu rosnącym lub opadającym. Dla uniknięcia konfuzji mówi się o pierwszym (ledaing) zboczu i od drugim (trailing).
- MOSI (Master Out Slave In) ta linia służy do przesyłania bitów przez mastera
- MISO (Master In Slave Out) ta linia służy do komunikacji w drugą stronę, od slave do mastera

Obecność linii MOSI oraz MISO daje protokołowi charakter full duplex (przesył danych w dwóch kierunkach jednocześnie)

SPI zużywa mniej energii od i2c, nie wymaga podtrzymujących rezystorów. Linia Chip select również przyczynia się do oszczędności energetycznych. Tylko urządzenie, które zostało wskazane przez linię Chip Select będzie nasłuchiwało danych na magistrali, reszta urządzeń może pozostać nieaktywna. Architektura linii chip select różni się od innych protokołów, między innymi i2c, dzięki niej urządzenia podpięte do magistrali nie muszą nasłuchiwać adresów paczek wysyłanych przez interfejs.

- *Przykładowe zastosowania w praktyce*

SPI jest stosowany w systemach wbudowanych gdzie wymagana jest szybka komunikacja między mikrokontrolerem a różnymi urządzeniami peryferyjnymi. Przykładem może być komunikacja mikrokontrolera z kartą SD właśnie za pomocą protokołu SPI.

5.3. Protokoły sieciowe

5.3.1. WebSockets

- *Cel i zastosowanie protokołu*

Websockets to protokół, który pozwala na utrzymanie stałego połączenia między klientem a serwerem, co umożliwia wymianę danych w czasie rzeczywistym. Dzięki temu aplikacje mogą działać szybciej i bardziej płynnie.

- *Opis działania protokołu*

W przeciwieństwie do protokołu HTTP nie trzeba nawiązywać osobnego połączenia przy każdym żądaniu. Po połączeniu otwiera się kanał, którym można wysyłać dane w pełnym

dupleksie (w obie strony na raz) i zostaje on otwarty aż do momentu przerwania sesji przez klienta lub server. Pierwsze zapytanie inicjalizujące websocket wysłane jest protokołem http, który w nagłówku ma informacje o zamiarze użycia protokołu websocket. Następnie po three-way handshake otiwera się trwałe połączenie po protokole TCP/IP. Websocket idealnie nadaje się do komunikacji typu realtime. Jest lżejszy i szybszy od http. WebSocket wykorzystywany jest w aplikacjach internetowych, które wymagają częstego odświeżania, działania w czasie rzeczywistym.

- *Przykładowe zastosowania w praktyce*

WebSockets znajduje zastosowanie w komunikatorach internetowych, gdzie konieczna jest ciągła i natychmiastowa wymiana wiadomości między użytkownikami.

6. Technologie sieciowe

6.1. Interfejsy

6.1.1. Ethernet

Ethernet to Standard stosowany w sieciach komputerowych. Odpowiada za najniższe warstwy modelu OSI, warstwę łącza danych i warstwę fizyczną. Ethernet ma największe zastosowanie w sieciach LAN. Pieczęć nad rozwojem technologii ma organizacja IEEE (Institute of Electrical and Electronics Engineers) [2]. Obecnie w sieciach Ethernet najczęściej korzysta się z kabla typu skrętka lub światłowodu, oraz topologii gwiazdy. Istnieją różne standardy Ethernet między innymi fastEthernet, gigabitEthernet czy 10gigabitEthernet, różnią się one przepustowością i maksymalną odległością na jaką przesyłane mogą być dane [8].

6.1.2. RS-232

RS-232 to interfejs transmisji szeregowej. Dziś rzadko spotykany. W zastosowaniach codziennych takich jak połączenie komputera z drukarką został wyparty przez zdecydowanie efektywniejsze USB. W przemyśle chętniej korzysta się z interfejsu RS-485 który umożliwia, między innymi, przesyłanie danych na znacznie większą odległość. [3] Główne wady RS-232 to maksymalna długość kabla 15m, oraz niska prędkość przesyłu danych, 20kB/s.

6.1.3. RS-485

RS-485 to młodszy szybszy brat RS-232. Co więcej może komunikować jednego mastera z wieloma slave'ami na raz (maksymalnie z 32), korzystając np. z metody przesyłu danych zwanej **broadcasting** która polega na jednoczesnym wysłaniu informacji od mastera do wszystkich urządzeń typu slave. Długoś kabla może sięgać do 1200m.

6.2. Firewall

Wszystkie projekty wykonano w sieci lokalnej LAN dystrybuowanej przez router WiFi w moim domu. Przy próbie łączenia się urządzeń przez sieć WAN na przeszkodzie staje usługa Firewall.

Oczywiście, z zainteresowaniem próbowałem obejść tą zaporę. Istnieje parę sposobów. Pierwszy Port Forwarding polega na skonfigurowaniu naszego routera tak aby pozwalał na połączenia z zewnątrz na konkretnym porcie i kierował je do konkretnego urządzenia. W sieci WAN wszystkie urządzenia podłączone do naszego routera mają ten sam adres IPv4, stąd konieczność skonfigurowania na routerze adresu docelowego. Niestety na drodze mogą stać kolejne przeszkody. Połączenia mogą być odrzucane po stronie dostawcy internetu, jeżeli ten korzysta z techniki CGNAT (Carrier-Grade NAT), która polega na translacji adresów IPv4. W rzeczywistości oznacza to że wielu klientów dostawcy ma ten sam publiczny adres IP, więc nie możemy sami konfigurować reguł dostępności poszczególnych portów. Tak było w moim przypadku.

7. Podsumowanie

W przebiegu tej pracy poznałem podstawy komunikacji pomiędzy procesami, oraz nabyłem wiedzę na temat popularnych systemów wbudowanych. Zrozumiałem warstwowy model, którym opisujemy protokoły. Nauczyłem się pisać programy umożliwiające komunikację. Poznałem podstawy sieci LAN i WAN. Dzięki szczegółowej analizie poszczególnych protokołów udało mi się zauważyć pewne zasady, którymi się rządzą. Protokół projektowany jest w myśl pewnego zastosowania, jego cechy uzależnione są od tego gdzie ma spełniać swoje zadanie. Stąd np. sygnał różnicowy w Profibus i CAN ma zapobiec zakłóceniom, które są charakterystyczne dla przemysłu i motoryzacji. Kolejnym przykładem może być lekkość MQTT, który ma być wykorzystany przez urządzenia o słabej przepustowości. Protokoły często łączone są w stosy. Tak np, FTP umożliwia proste i przyjazne dla użytkownika wysyłanie plików (warstwa aplikacji), ale żeby zapewnić niezawodowość dane z FTP przechodzą jeszcze przez protokół TCP który zapewnia że pliki nie zostaną zmodyfikowane po drodze lub nie przepadną. Fizyczną transmisją zajmują się interfejsy. Każdy protokół projektowany jest w oparciu o interfejs którym ma być przekazywany.

Wyróżnić można dwie główne architektury, modele, według których projektowane są protokoły. Pierwszy jest Klient-Serwer, drugi Master Slave. Spotykamy również inne protokoły nie korzystające z tych dwóch modeli. W CAN nie ma urządzenia nadrzędnego, w MQTT mamy broker, w którym można subskrybować oraz publikować dane tematy.

Różnorodność poszczególnych protokołów wydaje się zatem nieunikniona, aczkolwiek próby ujednolicania komunikacji takie jak np. OPC UA są obiecującą perspektywą, mogącą uprościć zagadnienie komunikacji.

Bibliografia

Książki:

[1] D. P. Kothari, S. K. Vasudevan, R. M. D. Sundaram, and N. Murali, *Embedded Systems*, 2nd ed. New Academic Science, 2014.

[2] A. S. Tanenbaum, *Sieci komputerowe*. Helion, 2010.

[3] J. Ułasiewicz, *Systemy czasu rzeczywistego QNX6 Neutrino*. Wydawnictwo BTC, 2007.

Źródła internetowe:

[4] Forbot, "Blog o elektronice i programowaniu," n.d. [Online]. Available: https://forbot.pl/blog

[5] Modbus Organization, "Modbus Application Protocol Specification V1.1b3," 2006. [Online]. Available: https://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf

[6] Pasja Informatyki, "Oficjalna strona internetowa," n.d. [Online]. Available: https://pasja-informatyki.pl

[7] RS Elektronika, "Kanał YouTube," n.d. [Online]. Available: https://www.youtube.com/c/RSElektronika

[8] D. Stelmach, "Ethernet," Pasja Informatyki, n.d. [Online]. Available: https://pasja-informatyki.pl/sieci-komputerowe/ethernet/

[9] D. Stelmach, "Model TCP/IP oraz ISO/OSI," Pasja Informatyki, n.d. [Online]. Available: https://pasja-informatyki.pl/sieci-komputerowe/model-tcp-ip-iso-osi

[10] D. Szymański, "Kurs Raspberry Pi, projekty – #5 – serwer www (PHP, MySQL)," Forbot, n.d. [Online]. Available: https://forbot.pl/blog/kurs-raspberry-pi-projekty-prosty-serwer-www-id27882